



الجمهورية العربية السورية

وزارة التعليم العالي والبحث العلمي

الجامعة الوطنية الخاصة

كلية الهندسة – قسم هندسة الحاسوب

مشروع أعد لنيل درجة الإجازة في هندسة الحاسوب بعنوان:

تصميم وتطوير مساعد برمجي لتحليل الشيفرات البرمجية

وتحسين جودتها

إعداد الطالبتين:

إليسا محمد حيان الأخرس

هبة مهند النكدلي

إشراف الدكتور:

علي ياسين

العام الدراسي 2025 - 2026

1	الفصل الأول: المقدمة.....
2	1-1 خلفية المشروع.....
3	2-1 المشكلة التي يعالجها المشروع.....
4	3-1 الأهداف.....
6	4-1 الأهمية.....
7	5-1 الجهات والفئات المستفيدة من مخرجات الأطروحة.....
9	6-1 الدراسة المرجعية والمقارنة (Benchmarking and Comparative Study).....
13	2 الفصل الثاني: التقنيات والأدوات المستخدمة.....
14	1-2 المقدمة.....
14	2-2 تقنيات الخادم (Server-Side Technologies).....
14	1-2-2 Node.js.....
16	2-2-2 وحدات Node.js المدمجة.....
17	3-2 تقنيات الواجهة الأمامية (Frontend Technologies).....
17	1-3-2 HTML5.....
18	2-3-2 CSS3.....
19	3-3-2 JavaScript (Vanilla JS).....
20	4-2 تقنيات المشاريع المولدة (Generated Projects Technologies).....

20	Node.js مشاريع 1-4-2
20	Python مشاريع 2-4-2
21	(Static Projects) المشاريع الثابتة 3-4-2
21	Flutter مشاريع 4-4-2
22	(Deployment and Launch Tools) أدوات التشغيل والنشر 5-2
22	Batch Scripts (ملفات .bat) 1-5-2
23	opencode / EHCode 2-5-2
24	6-2 تقنيات الذكاء الاصطناعي والمساعدة البرمجية (AI and Coding Assistant)
24	(Technologies 24
24	opencode 1-6-2 (وكيل البرمجة بالذكاء الاصطناعي).
24	Model Context Protocol (MCP) بروتوكول 2-6-2
25	(Evaluation and Monitoring Tools) أدوات التقييم والمراقبة 7-2
25	Playwright 1-7-2
25	Sisyphus 2-7-2
27	3 الفصل الثالث: تحليل وتصميم النظام.
28	1-3 تمهيد
28	2-3 تحليل المتطلبات

28	1-2-3 المتطلبات الوظيفية
30	2-2-3 المتطلبات غير الوظيفية.....
31	3-3 تصميم هيكلية النظام.....
32	4-3 تصميم حالات الاستخدام.....
35	5-3 تصميم التفاعلات الديناميكية.....
35	1-5-3 مخطط النشاط - تشغيل مشروع.....
36	2-5-3 مخطط التسلسل - فتح مساعد EHCode.....
38	6-3 تصميم إدارة الحالة والعمليات.....
41	4 الفصل الرابع: تنفيذ النظام والتطبيق العملي.....
42	1-4 المقدمة.....
42	2-4 إدارة المشاريع البرمجية.....
48	3-4 تحويل المشاريع من PHP إلى Laravel.....
50	4-4 محرر الأكواد الذكي.....
56	5-4 المساعد البرمجي الذكي EHCode.....
61	6-4 مقارنة أداء نماذج الذكاء الاصطناعي.....
67	5 الفصل الخامس: النتائج والآفاق المستقبلية.....

16	الشكل 2-1 آلية عمل Node.js
18	الشكل 2-2 تقنية iframe في HTML
25	الشكل 2-3 آلية عمل Playwrite
31	الشكل 3-1 مخطط هيكلية العمل
34	الشكل 3-2 مصمم حالات الاستخدام
35	الشكل 3-3 مخطط النشاط
37	الشكل 3-4 مخطط التسلسل
39	الشكل 3-5 مخطط الحالة
43	الشكل 4-1 الواجهة الرئيسية لإدارة واستعراض المشاريع البرمجية في منصة EHCode
46	الشكل 4-2: تشغيل مشروع منصة التدوين (Blog Platform) داخل منصة EHCode
	الشكل 4-3 تشغيل مشروع الكرة الأرضية ثلاثية الأبعاد (D Earth Globe3) داخل منصة EHCode
47	
48	الشكل 4-4 واجهة محرك تحويل المشاريع البرمجية من PHP إلى Laravel
51	الشكل 4-5 الواجهة الرئيسية لمحرر الأكواد الذكي المدعوم بالذكاء الاصطناعي
53	الشكل 4-6 كتابة الشيفرة البرمجية داخل محرر الأكواد الذكي
54	الشكل 4-7 تحويل الشيفرة البرمجية من لغة Python إلى لغة C++ باستخدام الذكاء الاصطناعي
55	الشكل 4-8 تحليل الشيفرة البرمجية باستخدام الذكاء الاصطناعي
57	الشكل 4-9 الواجهة الرئيسية للمساعد البرمجي EHCode Web
58	الشكل 4-10 واجهة المحادثة مع المساعد البرمجي EHCode Web
60	الشكل 4-11 واجهة المساعد البرمجي EHCode ضمن بيئة سطر الأوامر (CLI)

الشكل 12-4 واجهة مقارنة نماذج الذكاء الاصطناعي وفق مؤشرات الأداء ومعايير جودة البرمجيات.

62

الاختصارات

الاختصار	المصطلح باللغة الإنجليزية	المصطلح باللغة العربية
API	Application Programming Interface	واجهة برمجة التطبيقات
CLI	Command Line Interface	واجهة سطر الأوامر
CORS	Cross-Origin Resource Sharing	مشاركة الموارد بين المصادر المختلفة
CSS	Cascading Style Sheets	صفحات الأنماط المتتالية
DOM	Document Object Model	نموذج كائن المستند
DSL	Domain-Specific Language	لغة خاصة بالمجال
FSM	Finite State Machine	آلة الحالات المحدودة
GUI	Graphical User Interface	واجهة المستخدم الرسومية
HTML	HyperText Markup Language	لغة ترميز النصوص التشعبية
HTTP	HyperText Transfer Protocol	بروتوكول نقل النصوص التشعبية
IDE	Integrated Development Environment	بيئة التطوير المتكاملة
I/O	Input/Output	الإدخال/الإخراج
JSON	JavaScript Object Notation	تدوين كائنات جافا سكريبت
LLM	Large Language Model	النموذج اللغوي الكبير
MCP	Model Context Protocol	بروتوكول سياق النموذج
PID	Process Identifier	معرف العملية
REST	Representational State Transfer	نقل الحالة التمثيلي
SDK	Software Development Kit	مجموعة أدوات تطوير البرمجيات
SQL	Structured Query Language	لغة الاستعلامات البنوية
TUI	Text User Interface	واجهة المستخدم النصية
UML	Unified Modeling Language	لغة النمذجة الموحدة
URI	Uniform Resource Identifier	معرف الموارد الموحد
URL	Uniform Resource Locator	محدد الموارد الموحد
UTF	Unicode Transformation Format	تنسيق تحويل يونيكود
YAML	YAML Ain't Markup Language	لغة ليست للترميز

يشهد مجال هندسة البرمجيات تحولاً نوعياً مع التزايد المطرد في استخدام نماذج الذكاء الاصطناعي التوليدية لأتمتة كتابة الأكواد البرمجية، مما يستدعي تطوير مناهج ومنصات موضوعية لتقييم ومقارنة أداء هذه النماذج في مهام برمجية واقعية ومتنوعة. يهدف هذا البحث إلى تصميم وبناء منصة متكاملة تحت مسمى "EHCode Hub"، تعمل كحاضنة تقييم (Benchmark Harness) لمقارنة قدرات ثلاثة نماذج رائدة في توليد الكود، وهي: GLM-5.1 من Zhipu AI، وDeepSeek V4 Pro، و MiMo V2.5 من Xiaomi Pro. ولتحقيق هذا الهدف، تم تصميم عشرين مهمة برمجية موحدة تتوزع على عشر فئات تطويرية رئيسية تشمل تطبيقات الويب المتكاملة، وواجهات برمجة التطبيقات، والتطبيقات الواجهة البحتة، وتطبيقات بايثون، والتجارة الإلكترونية، وتصور البيانات، والنمذجة ثلاثية الأبعاد، وتطبيقات الجوال باستخدام Flutter، وأدوات الأتمتة. كما تم بناء خادم عارض تفاعلي باستخدام تقنية Node.js معتمداً حصراً على وحداته المدمجة وبلا أي اعتماديات خارجية، إلى جانب واجهة أمامية ديناميكية، وتم دمج مساعد برمجي ذكي (EHCode/opencode) عبر نافذة iframe داخل المنصة، مع مشغل موحد يقلع كافة المكونات في الخلفية بنقرة واحدة.

أظهرت النتائج أن النماذج الثلاثة أنتجت مجتمعة ستين مشروعاً، توزعت بين عشرة مشاريع Node.js وستة مشاريع بايثون وأربعين مشروعاً ثابتاً وأربعة مشاريع Flutter حقيقية. كما أظهر النموذج GLM-5.1 تفوقاً ملحوظاً من حيث الالتزام الكامل بالمواصفات، حيث أنتج تطبيقات Flutter كما هو مطلوب، وسجل أعلى نسبة توثيق بخمسة مشاريع احتوت على ملف README من أصل عشرين، مما جعله النموذج الموصى به للاستخدام العام. في المقابل، أظهر نموذج DeepSeek V4 Pro انحرافاً واضحاً عن المتطلبات، حيث أنتج صفحات HTML بدلاً من تطبيقات Flutter للمهمتين المخصصتين للجوال، كما كان الأضعف في التوثيق بمشروع واحد فقط، بينما أظهر نموذج MiMo V2.5 Pro أداءً متوازناً في المشاريع الواجهة. أثبتت المنصة كفاءة عالية في إدارة العمليات وتحرير المنافذ وتشغيل المشاريع تلقائياً، ونجح دمج المساعد البرمجي دون عوائق أمنية بفضل غياب ترويسات منع التضمين.

تُخَلِّص هذه الدراسة إلى أن البنية المعمارية "بلا اعتماديات" أثبتت جدواها في سهولة النقل والتشغيل، وتقدم المنصة إسهاماً عملياً ملموساً في مجال تقييم نماذج الذكاء الاصطناعي التوليدية، مع التوصية باستخدام نموذج GLM-5.1 للتطبيقات العامة، ونموذج DeepSeek للمشاريع الخلفية، ونموذج MiMo للمشاريع

الواجهية البحتة، مع التأكيد على ضرورة إكمال تسجيل الدرجات وتحديث بيانات النتائج كخطوة تالية لإتمام عملية التقييم الكمي.

الكلمات المفتاحية: الذكاء الاصطناعي التوليدي، تقييم الأكواد البرمجية، البنشمارك، معالجة اللغات الطبيعية، هندسة البرمجيات، النماذج اللغوية الكبيرة، توليد الكود.

Abstract

The field of software engineering is undergoing a paradigm shift with the growing reliance on generative artificial intelligence models for automating code synthesis, which necessitates the development of objective methodologies and platforms to evaluate and compare the performance of these models across diverse and realistic programming tasks. This research aims to design and build an integrated platform named "EHCode Hub", serving as a benchmark harness to compare the capabilities of three leading code generation models: GLM-5.1 by Zhipu AI, DeepSeek V4 Pro, and MiMo V2.5 Pro by Xiaomi. To accomplish this objective, a set of twenty standardized programming tasks was meticulously designed, spanning ten major development categories, including full-stack web applications, RESTful APIs, pure frontend interfaces, Python applications, e-commerce systems, data visualization, 3D graphics, mobile applications using Flutter, and CLI automation tools. Furthermore, an interactive viewer server was constructed using Node.js, relying exclusively on its built-in modules with zero external dependencies, alongside a dynamic frontend interface; an intelligent coding assistant (EHCode/opencode) was seamlessly embedded via an iframe within the platform, complemented by a unified launcher that automatically starts all components in the background with a single click.

The experimental results revealed that the three models collectively produced sixty projects, distributed across ten Node.js projects, six Python projects, forty static projects, and four genuine Flutter projects. The GLM-5.1 model demonstrated remarkable superiority in terms of full specification adherence, successfully generating authentic Flutter applications as required, and achieved the highest documentation rate with five projects containing a README.md file out of twenty, rendering it the recommended model for general-purpose usage. Conversely, the DeepSeek V4 Pro model exhibited a significant deviation from the requirements by generating static HTML pages instead of Flutter/Dart applications for the two dedicated mobile tasks, and it had the weakest documentation with merely one README file, while the MiMo V2.5 Pro model displayed balanced performance in frontend-oriented projects. The platform proved highly efficient in process management, automated port releasing, and seamless project launching, and the integration of the coding assistant succeeded without security barriers due to the absence of frame-blocking headers.

This study concludes that the zero-dependency architectural approach has proven its viability in terms of portability and ease of deployment, and the platform offers a tangible practical contribution to the field of generative AI model evaluation. It recommends the use of GLM-5.1 for general applications, DeepSeek for backend-intensive projects, and MiMo for pure frontend tasks, while emphasizing the necessity of completing the scoring registration and updating the results data as a subsequent step to finalize the quantitative evaluation process.

Keywords: Generative AI, Code Evaluation, Benchmarking, Natural Language Processing, Software Engineering, Large Language Models, Code Generation.

الفصل الأول: المقدمة

1-1 خلفية المشروع

شهدت السنوات الأخيرة تقدماً هائلاً في مجال الذكاء الاصطناعي التوليدي، ولا سيما في تطوير النماذج اللغوية الكبيرة (Large Language Models – LLMs) القادرة على فهم وتوليد الأكواد البرمجية بدقة وكفاءة متزايدة. لقد أحدثت هذه النماذج تحولاً جوهرياً في طريقة تطوير البرمجيات، حيث باتت تستخدم على نطاق واسع في أتمتة كتابة الأكواد، وتحسين إنتاجية المبرمجين، ودعم عمليات التعلم البرمجي، مما ساهم في تقليل الأخطاء البشرية وتسريع دورات التطوير بشكل ملحوظ [1].

تمثل مهام توليد الكود البرمجي (Code Generation) واحدة من أكثر التطبيقات الواعدة للنماذج اللغوية الكبيرة، إذ تتيح للمطورين والمؤسسات إمكانية الانتقال من المواصفات الوظيفية إلى أكواد تنفيذية جاهزة في زمن قياسي. وقد استقطبت هذه النماذج اهتماماً كبيراً من الباحثين والشركات على حد سواء، مما أدى إلى ظهور العديد من النماذج الرائدة التي تتنافس في تقديم أعلى مستويات الدقة والفهم السياقي للغات البرمجة المختلفة [2]. غير أن التحدي الأكبر لا يكمن في تطوير النماذج فحسب، بل في قدرة المجتمع البحثي والصناعي على تقييمها بشكل موثوق وعادل لمقارنة قدراتها الحقيقية.

إن عملية تقييم النماذج اللغوية الكبيرة في توليد الكود تتطلب أكثر من مجرد اختبارات بسيطة؛ فهي تحتاج إلى مناهج علمية رصينة تعتمد على مهام برمجية متنوعة تغطي مختلف جوانب التطوير، بدءاً من التطبيقات الواجهية البسيطة وصولاً إلى الأنظمة الخلفية المعقدة وتطبيقات الجوال والتصور ثلاثي الأبعاد. وقد أشارت الدراسات الحديثة إلى أن التقييم الموضوعي يجب أن يأخذ في الاعتبار عدة معايير متكاملة مثل قابلية التشغيل الفعلية، وجودة الكود من حيث التنظيم والنمطية، والجانب البصري، ومدى اكتمال المتطلبات الوظيفية [3]. ومع تزايد عدد النماذج وتنوع قدراتها، برزت الحاجة الملحة إلى تطوير منصات

وأدوات قياسية تعمل كـ "حاضنات تقييم" (Benchmark Harnesses) توفر بيئة موحدة لاختبار هذه النماذج في ظروف متكافئة.

علاوة على ذلك، فإن دمج المساعدات البرمجية الذكية في بيئات التطوير أصبح اتجاهًا متناميًا، حيث أثبتت هذه الأدوات قدرتها على تعزيز إنتاجية المطورين وتقديم اقتراحات ذكية في الوقت الفعلي [5]. لذلك، فإن تصميم منصة لا تقتصر على التقييم فقط، بل تدمج أيضاً مساعداً برمجياً تفاعلياً، يمثل نقلة نوعية في كيفية تفاعل المطورين مع أدوات الذكاء الاصطناعي، مما يعكس التوجه الحديث نحو بيئات التطوير المتكاملة التي تجمع بين التحليل والمقارنة والدعم التفاعلي في مكان واحد.

1-2 المشكلة التي يعالجها المشروع

على الرغم من التوسع الكبير في استخدام النماذج اللغوية الكبيرة في توليد الأكواد البرمجية، إلا أن هناك فجوة بحثية وعملية واضحة تتمثل في غياب منصة تقييمية موحدة وشاملة تسمح بمقارنة أداء هذه النماذج في مهام برمجية واقعية ومتنوعة تحت ظروف متكافئة. فمعظم التقييمات الحالية إما أن تكون محدودة النطاق، أو تركز على معايير ضيقة لا تعكس التعقيد الحقيقي لمشاريع البرمجيات، أو تعتمد على معايير آلية لا ترتبط بالجودة الفعلية للمخرجات من وجهة نظر المطورين والمختبرين [2].

وتتمثل المشكلة الأساسية في أن النماذج المختلفة قد تتفوق في مجالات معينة وتضعف في مجالات أخرى، إلا أن غياب أداة مقارنة موحدة تجعل من الصعب على المطورين والمؤسسات اختيار النموذج الأنسب لاحتياجاتهم الخاصة. فبعض النماذج قد تبرع في المهام الواجبة، في حين تتفوق نماذج أخرى في البنية الخلفية أو تطبيقات الجوال، مما يستدعي وجود أداة تسمح بعرض المشاريع جنباً إلى جنب مع تحليل نقاط القوة والضعف لكل نموذج [4]. كما أن غياب التوثيق الكافي للمشاريع المولدة يعد مشكلة إضافية، حيث يصعب على المطورين فهم كيفية تشغيل المشاريع أو اختبارها دون دليل واضح.

بالإضافة إلى ذلك، فإن عمليات تشغيل المشاريع المولدة يدوياً تتطلب جهداً كبيراً في تثبيت الاعتماديات، وتحرير المنافذ، والتعامل مع بيانات التشغيل المختلفة، مما يضيع وقتاً ثميناً كان يمكن استثماره في التقييم الفعلي. كما أن معظم بيانات التقييم الحالية لا تدمج مساعداً برمجياً تفاعلياً يمكن للمطور الاستعانة به أثناء عملية التقييم لفهم الكود أو تحسينه، مما يفصل بين عملية التقييم وعملية التطوير الفعلية [5].

وعليه، فإن المشكلة التي يعالجها هذا المشروع هي: الحاجة الملحة إلى منصة متكاملة تعمل كحاضنة تقييم (Benchmark Harness) لمقارنة النماذج اللغوية الكبيرة في توليد الكود البرمجي، تجمع بين الفحص التلقائي للمشاريع، وعرضها التفاعلي، وتشغيلها الآلي، ودليل توثيقي لكل مشروع، بالإضافة إلى دمج مساعد برمجي ذكي لتوفير بيئة تقييم شاملة وموحدة، بحيث تمكن المطورين والباحثين من اتخاذ قرارات مستنيرة بشأن النموذج الأنسب لتطبيقاتهم المختلفة.

1-3 الأهداف

يهدف هذا المشروع إلى تحقيق مجموعة من الأهداف الرئيسية، تتكامل فيما بينها لتقديم حل عملي وشامل لمشكلة تقييم ومقارنة نماذج توليد الكود البرمجي، ويمكن تفصيل هذه الأهداف على النحو التالي:

1. تصميم مجموعة موحدة من المهام البرمجية: بناء 20 مهمة برمجية (بنشمارك) تغطي عشر فئات تطويرية رئيسية، تشمل تطبيقات الويب المتكاملة، وواجهات برمجة التطبيقات، والتطبيقات الواجهية، وتطبيقات بايثون، والتجارة الإلكترونية، وتصور البيانات، والنمذجة ثلاثية الأبعاد، وتطبيقات الجوال (Flutter)، وأدوات الأتمتة، بحيث تمثل عينة متنوعة وشاملة لمتطلبات التطوير الواقعية [3].

2. بناء خادم عارض تفاعلي: تطوير خادم ويب خفيف باستخدام Node.js، يعتمد على الوحدات المدمجة فقط (بلا اعتماديات خارجية)، قادر على فحص المشاريع المخزنة تلقائياً، واكتشاف نوع

كل مشروع (Node.js، Python، Static، Flutter)، ونقطة الدخول، والمنفذ المطلوب، وعرضها ضمن واجهة مستخدم تفاعلية بديهية [5].

3. توفير آلية تشغيل تلقائي للمشاريع: تمكين المستخدم من تشغيل أي مشروع مباشرة من خلال الواجهة، مع إدارة تلقائية لعمليات التثبيت (npm install / pip install)، وتحرير المنافذ، وإطلاق الخوادم في الخلفية، وعرض المخرجات داخل نافذة iframe دون تدخل يدوي، مما يسهل عملية اختبار قابلية التشغيل الفعلية [2].

4. بناء دليل توثيقي تفاعلي لكل مشروع: توليد دليل تلقائي لكل مشروع يتضمن المتطلبات المسبقة، وأوامر التثبيت والتشغيل، وأوامر التحقق، والنتائج المتوقعة، وقائمة الاعتماديات، وقائمة الملفات، ومقتطفات من ملف README إن وجد، لتسهيل فهم المشاريع واختبارها، ومعالجة مشكلة غياب التوثيق في المخرجات المولدة [4].

5. دمج مساعد برمجي ذكي (EHCode/opencode): دمج مساعد كود تفاعلي عبر نافذة iframe داخل المنصة نفسها، بحيث يتمكن المستخدم من الاستعانة بالمساعد لفهم الكود أو طرح الأسئلة البرمجية أثناء عملية التقييم، مما يوحد بيئة العمل ويجعل المنصة مركزاً متكاملًا للمقارنة والتعلم والدعم البرمجي [1].

6. توفير آلية مقارنة موضوعية: بناء جدول مقارنة تفاعلي يعرض النماذج جنباً إلى جنب مع مؤشرات الأداء الرئيسية (عدد المشاريع، وجودة التوثيق، والتنوع التقني)، مع تقديم توصية تلقائية بالنموذج الأنسب بناءً على معايير محددة، مما يساعد المطورين على اختيار النموذج الأمثل لتطبيقاتهم.

7. تسهيل عملية النشر والتشغيل: توفير مشغل موحد (start.bat) على سطح المكتب يقلع جميع مكونات المنصة (العارض ومساعد الكود) في الخلفية بنقرة واحدة، مع مشغل إيقاف (stop.bat)

لإعادة ضبط البيئة بسهولة، مع ضمان التوافق مع بيئة Windows وتحقيق مبدأ "التشغيل الفوري" دون تعقيدات في الإعداد.

4-1 الأهمية

تتجلى أهمية هذا المشروع على عدة مستويات؛ أكاديمية وبحثية وعملية، يمكن إجمالها في النقاط التالية:

- أولاً: الأهمية البحثية والأكاديمية: يمثل هذا المشروع إسهاماً عملياً في مجال تقييم النماذج اللغوية الكبيرة، حيث يقدم منهجية ومنصة مفتوحة يمكن للباحثين الاستفادة منها في تقييم نماذج جديدة مستقبلاً. فبدلاً من الاعتماد على معايير آلية مجردة، تتيح المنصة التقييم البصري والتفاعلي للمخرجات، مما يتوافق مع التوجهات الحديثة التي تدعو إلى تقييم أكثر شمولية يراعي جوانب جودة الكود، وقابلية التشغيل، والخبرة البصرية، واكتمال المتطلبات [4]. كما أن تصميم عشرين مهمة موزعة على عشر فئات يقدم نموذجاً مرجعياً يمكن تحديثه وتطويره ليشمل مهام جديدة، مما يثري المكتبة البحثية في هذا المجال [3].

- ثانياً: الأهمية العملية للمطورين والمؤسسات: توفر المنصة أداة عملية تمكن المطورين وفرق العمل ومدراء المشاريع من مقارنة النماذج المختلفة بناءً على مشاريع حقيقية، وليس فقط على مؤشرات نظرية. فمن خلال المنصة، يمكن للمستخدم رؤية الكود الذي يولده كل نموذج، وتشغيله مباشرة، وتقييم جودته وشكله ومدى اكتماله، مما يساعد في اتخاذ قرارات مستنيرة بشأن اعتماد نموذج معين في مشاريعهم الإنتاجية. وهذا له أثر مباشر على تحسين إنتاجية الفرق البرمجية وجودة مخرجاتها [2].

- ثالثاً: تعزيز الشفافية والموضوعية في تقييم النماذج: إن توحيد المهام وظروف التشغيل للنماذج الثلاثة يضمن تكافؤ الفرص، مما يعزز الشفافية في عملية المقارنة. فبدلاً من الاعتماد على

إعلانات الشركات المنتجة، يمكن للمطورين التحقق بأنفسهم من أداء كل نموذج في سياق مشابه لسياق عملهم الفعلي. كما أن دمج المساعد البرمجي يمنح المستخدم القدرة على طرح أسئلة توضيحية والتفاعل مع المخرجات، مما يضيف طبقة إضافية من الفهم والتقييم النقدي [1].

- رابعاً: الابتكار التقني في بنية التقييم: يكمن الابتكار في تبني فلسفة "صفر اعتماديات" (Zero Dependencies) في بناء خادم العارض، مما يجعله خفيف الوزن، سريع النقل، وسهل النشر على أي جهاز دون الحاجة إلى تثبيت حزم إضافية معقدة. كما أن التكامل بين التقييم ومساعد الكود في منصة واحدة هو نهج جديد نسبياً، حيث تجمع الأداة بين ثلاثة أدوار كانت منفصلة سابقاً: التقييم، التشغيل، والدعم البرمجي، مما يقلل من الاحتكاك الذهني والانتقال بين الأدوات المختلفة أثناء عملية التقييم [5].

- خامساً: دعم بيئة التطوير العربية: يوفر المشروع واجهة عربية كاملة، مما يسهم في تعريب مصطلحات وأدوات الذكاء الاصطناعي التطبيقي، ويسهل على المطورين والباحثين العرب التفاعل مع هذه التقنيات المتقدمة بلغتهم الأم، مما يوسع قاعدة المستفيدين ويعزز المحتوى التقني العربي في هذا المجال الحيوي.

1-5 الجهات والفئات المستفيدة من مخرجات الأطروحة

يستهدف هذا المشروع مجموعة واسعة من الفئات التي يمكن أن تستفيد من المنصة بطرق مختلفة، ويمكن تحديد هذه الفئات على النحو التالي:

1. الباحثون الأكاديميون في مجال الذكاء الاصطناعي وهندسة البرمجيات: يستفيد الباحثون من المنصة كأداة مرجعية لتقييم النماذج اللغوية الكبيرة الجديدة، ومنهجية موثقة يمكن الاستناد إليها

في الأبحاث العلمية، مما يوفر لهم أرضية تجريبية خاضعة للرقابة لاختبار فرضياتهم حول أداء النماذج في توليد الكود [3].

2. مطورو البرمجيات وفرق التطوير: تعد المنصة أداة مساعدة قوية لمطوري البرمجيات لاختيار النموذج الأنسب لاحتياجاتهم اليومية. سواء كانوا يعملون في تطوير الواجهات، أو بناء واجهات برمجة التطبيقات، أو تطبيقات الجوال، يمكنهم معاينة قدرات كل نموذج في السياق الذي يهتمهم، بالإضافة إلى الاستعانة بالمساعد البرمجي لطرح الأسئلة والحصول على اقتراحات فورية [5].

3. مدراء المشاريع وقادة الفرق التقنية: تساعد المنصة مدراء المشاريع في اتخاذ قرارات استراتيجية بشأن تبني أدوات الذكاء الاصطناعي في فرقهم، حيث يمكنهم تقديم تقارير مرئية ومقارنة موضوعية للمخرجات بين النماذج المختلفة، مما يسهل عملية التبرير وإقناع أصحاب المصلحة باعتماد نماذج معينة لتحسين الإنتاجية وجودة المنتج النهائي [2].

4. المهندسون والمختصون في ضمان الجودة واختبار البرمجيات: يمكن لفريق الجودة والاختبار استخدام المنصة لتقييم جودة الكود المُولد تلقائياً من حيث قابليته للتشغيل، وخلوه من الأخطاء الواضحة، واكتمال المتطلبات، مما يمكنهم من وضع معايير داخلية لقبول أو رفض المخرجات المولدة آلياً ضمن عمليات التطوير المؤسسية [4].

5. المتعلمون وطلاب البرمجيات: تمثل المنصة أداة تعليمية قيّمة لطلاب علوم الحاسوب وهندسة البرمجيات، حيث تتيح لهم الاطلاع على حلول مختلفة لنفس المشكلة من نماذج متعددة، ومقارنة الأساليب والأكواد، والاستعانة بالمساعد البرمجي لفهم الكود أو تحسينه، مما يعزز مهاراتهم البرمجية وقدرتهم على التقييم النقدي للكود [1].

1-6 الدراسة المرجعية والمقارنة (Benchmarking and Comparative Study)

شهد مجال تقييم النماذج اللغوية الكبيرة في مهام توليد الكود البرمجي تطوراً متسارعاً خلال السنوات الأخيرة، مما أدى إلى ظهور عدد كبير من المعايير المرجعية (Benchmarks) والمنصات المقارنة التي تهدف إلى قياس أداء هذه النماذج بشكل موضوعي وقابل للتكرار. وقد أسهم هذا التوجه في ترسيخ منهجية علمية رصينة لتقييم النماذج، تقوم على توفير مهام برمجية موحدة، ومجموعات اختبار محددة، ومقاييس أداء قابلة للقياس الكمي. وقد أوضحت دراسة شاملة أجراها Chen وزملاؤه (2024) أن تقييم النماذج اللغوية الكبيرة في توليد الكود يتطلب أكثر من مجرد اختبار الصحة الوظيفية، بل يجب أن يشمل مقاييس متعددة الأبعاد مثل كفاءة الكود، وقابليته للقراءة، ومدى توافقه مع متطلبات المستخدمين الفعلية، مما يستدعي تطوير مناهج تقييم تجمع بين المعايير الآلية والمراجعة البشرية الخبيرة [5].

تُعد معايير HumanEval و MBPP من أبرز المعايير المرجعية وأكثرها استخداماً في هذا المجال، حيث تركزان على تقييم الصحة الوظيفية للمقطعات البرمجية الصغيرة بلغة Python، وتختبران قدرة النماذج على فهم الخوارزميات واللغات والمكتبات الأساسية. وقد تطورت هذه المعايير لاحقاً إلى نسخ موسعة مثل HumanEval+ و MBPP+ التي تزيد من تغطية الاختبارات للكشف عن حالات الحافة، وكذلك HumanEval Pro و MBPP Pro التي تُدخل مهام تركيبية تتطلب استدعاءات متعددة للدوال، مما يختبر قدرة النماذج على الاستدلال عبر خطوات توليد متعددة. وفي سياق أوسع، تشير دراسة استقصائية شملت 142 ورقة بحثية نشرت بين يناير 2020 ويونيو 2025 إلى وجود 156 معياراً مرجعياً فريداً تغطي 32 مهمة برمجية مختلفة، مع تركيز ملحوظ على مهمة توليد الكود التي خُصص لها 86 معياراً من هذه المعايير، مما يعكس الأهمية البالغة التي توليها الأوساط الأكاديمية والصناعية لهذه المهمة [6].

غير أن الدراسات المرجعية كشفت أيضاً عن قيود جوهرية في المعايير الحالية، أبرزها التحيز في مجموعات البيانات، وغياب بروتوكولات تقييم موحدة، وعدم كفاية التغطية للتحديات الواقعية التي تواجه المطورين في المشاريع البرمجية الكبيرة. كما أشارت دراسة تحليلية معمقة أجراها Molzam Sharifloo وزملاؤه

(2025) إلى أن المعايير المرجعية واللوحات المتصدرة (Leaderboards)، رغم انتشارها الواسع، تقدم رؤى محدودة حول المهام التي تفشل النماذج في حلها باستمرار، مما يعيق فهم القيود الحالية للنماذج وتوجيه تطوير نماذج أكثر قدرة. وقد كشف تحليلهم لـ 114 مهمة واجهت النماذج صعوبة متكررة في حلها عن أربعة أنماط متكررة من نقاط الضعف في النماذج، بالإضافة إلى تعقيدات شائعة في مهام المعايير نفسها تؤدي غالباً إلى الفشل [9]. وتؤكد هذه النتائج الحاجة الملحة إلى تطوير منصات تقييم لا تكتفي بتصنيف النماذج كمياً، بل تقدم تحليلاً نوعياً متعمقاً لنقاط القوة والضعف، وتوفر بيئة تفاعلية تسمح للمستخدمين بفحص المخرجات وتشغيلها وتقييمها في سياقات تحاكي التطوير الواقعي.

وفي هذا السياق، برز OpenCode كأحد أبرز الأدوات مفتوحة المصدر التي تسهم في سد الفجوة بين التقييم الأكاديمي والتطبيق العملي، حيث يُعد وكيل برمجة بالذكاء الاصطناعي تفاعلياً يعمل في الطرفية (Terminal) أو بيئات التطوير المتكاملة، ويدعم أكثر من 75 مزوداً لنماذج الذكاء الاصطناعي، بما في ذلك النماذج السحابية (Claude، GPT، Gemini) والنماذج المحلية (Ollama)، مما يمنح الباحثين والمطورين مرونة كاملة في اختيار النموذج المناسب دون تقييد بمزود واحد [10]. وقد تجاوز عدد نجوم OpenCode على GitHub 182 ألف نجمة، ويستعمله أكثر من 8 ملايين مطور شهرياً، مما يجعله أكثر وكيل برمجة ذكي مفتوح المصدر اعتماداً في التاريخ، وقد استُخدم على نطاق واسع في الأوساط الأكاديمية كمنصة لتطوير إضافات بحثية متخصصة تغطي دورة البحث الأكاديمي بأكملها، بدءاً من مراجعة الأدبيات وصولاً إلى إعداد مخطوطة جاهزة للمراجعة العلمية [11]، كما طوّرت له إضافات تتيح البحث التلقائي عن الأوراق العلمية في مستودعات مثل arXiv و OpenAlex دون الحاجة إلى مفاتيح API، مما يسهل على الباحثين عملية الاستشهاد والتحقق من المراجع [12].

كما يُستخدم OpenCode كأداة مرجعية في الدراسات المقارنة لوكلاء البرمجة بالذكاء الاصطناعي، حيث أشارت دراسات حديثة إلى أن أدوات مثل OpenCode و Claude Code تمثل جيلاً جديداً من الوكلاء يتجاوز مجرد إكمال الأكواد، ليؤسس لسير عمل "وكيلي" يتفاعل مع المطور داخل الطرفية وينفذ أوامر وملفات بشكل مستقل [13]. وقد استندت أوراق بحثية رائدة إلى OpenCode لتجسيد منهجيات متقدمة مثل "الترميز الجدلي" (Dialectical Autocoding)، وهو أسلوب يعتمد على تنافس تعاوني منظم بين وكلاء متخصصين (أطروحة-نقيض-تركيب) لمعالجة نمط فشل حرج في الوكلاء الأحاديين، ألا وهو "الإعلان المبكر عن النجاح" حيث يميل الوكيل الواحد الذي يكتب ويراجع كوده بنفسه إلى التغاضي عن الأخطاء [14]. ويعمل فريق OpenCode حالياً على إضافة دعم رسمي للتقييم على معيار SWE-bench الذي يقيس قدرة النماذج على حل مهام برمجية واقعية، مما سيزود الباحثين والمطورين برؤية دقيقة حول أداء الأداة في سياقات هندسية حقيقية، وليس فقط على معايير اصطناعية محدودة.

إن دمج OpenCode في منصات التقييم مثل منصة EHCCode Hub المقترحة في هذا المشروع يمثل نقلة نوعية، حيث يتيح للباحثين والمطورين ليس فقط مقارنة مخرجات النماذج المختلفة، بل أيضاً التفاعل معها وطرح الأسئلة التوضيحية والحصول على اقتراحات فورية لتحسين الكود، مما يحول عملية التقييم من مجرد مقارنة سلبية إلى تجربة تعلم وتطوير تفاعلية. وبذلك، يسهم هذا المشروع في معالجة القيود التي كشفت عنها الدراسات المرجعية السابقة، عبر توفير منصة تقييم شاملة تجمع بين الفحص الآلي، والتشغيل التفاعلي، والتوثيق التلقائي، والدعم البرمجي الذكي، في بيئة موحدة مفتوحة المصدر تعزز الشفافية والموضوعية في تقييم النماذج اللغوية الكبيرة، وتقدم إسهاماً عملياً ملموساً للمجتمع البحثي والصناعي على حد سواء.

الفصل الثاني: التقنيات والأدوات

المستخدمة

1-2 المقدمة

يعتمد نجاح أي مشروع برمجي على الاختيار الدقيق للتقنيات والأدوات التي تُستخدم في بنائه، إذ تحدد هذه الاختيارات مدى كفاءة النظام، وسهولة تطويره، وقابلية صيانتها، وقدرته على التكيف مع المتطلبات المتغيرة. وفي مشروع منصة EHCCode Hub، تمثل التحدي الأكبر في بناء نظام متكامل يجمع بين خادم ويب خفيف الوزن، وواجهة مستخدم تفاعلية، وآلية لإدارة العمليات وتشغيل المشاريع تلقائياً، مع دمج مساعد برمجي ذكي، كل ذلك مع الحفاظ على مبدأ "صفر اعتماديات" (Zero Dependencies) في المكونات الأساسية لتسهيل النقل والتشغيل على أي جهاز [16]. وقد تم اختيار كل تقنية بعناية لتحقيق أهداف محددة، مع مراعاة التوافق، والأداء، وسهولة الاستخدام، والاعتماد على معايير مفتوحة المصدر قدر الإمكان [17].

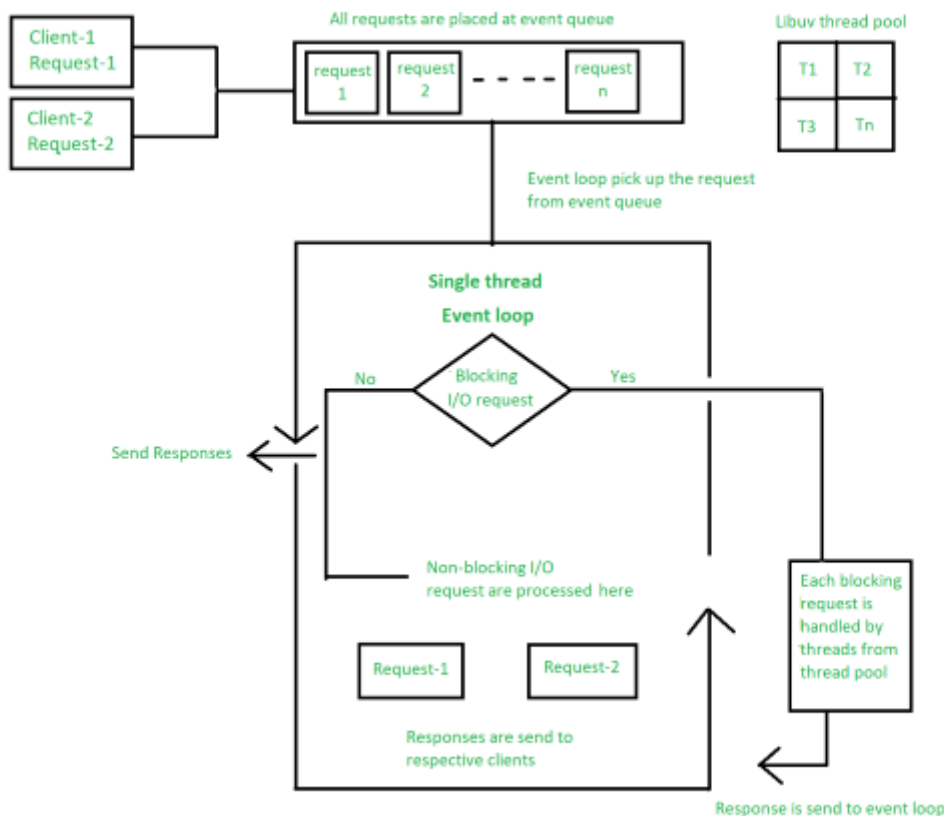
يتناول هذا الفصل استعراضاً مفصلاً لجميع التقنيات والمكتبات والأدوات المستخدمة في بناء المنصة، مع توضيح الدور الذي تؤديه كل منها، والمكان الذي استخدمت فيه، والأسباب التي دعت إلى اختيارها، وسيتم تقسيم التقنيات حسب المكونات الرئيسية للمنصة، بدءاً من تقنيات الخادم، مروراً بتقنيات الواجهة الأمامية، وانتهاءً بتقنيات المشاريع المولدة وأدوات التشغيل والنشر.

2-2 تقنيات الخادم (Server-Side Technologies)

Node.js 1-2-2

تُعد Node.js بيئة تشغيل JavaScript مفتوحة المصدر، مبنية على محرك V8 الخاص بمتصفح Google Chrome، وتسمح بتشغيل JavaScript على جانب الخادم (Server-Side) [16]. تم اختيار Node.js لتكون التقنية الأساسية لبناء خادم العارض (Viewer Server) في هذا المشروع، وذلك لما توفره من كفاءة وأداء عالٍ بفضل نموذج الإدخال/الإخراج غير المحظور (Non-blocking I/O) والحلقة

الزمنية (Event Loop)، مما يجعلها قادرة على التعامل مع عدد كبير من الطلبات المتزامنة بكفاءة عالية، وهو أمر ضروري لخدم واجهة ويب تفاعلية [17]. كما أن التوافق مع المشاريع المولدة بشكل عاملاً حاسماً، إذ أن العديد من المشاريع التي أنتجت النماذج الثلاثة كانت تعتمد على Node.js و Express.js، مما يضمن التناسق والسهولة في التعامل مع هذه المشاريع وتشغيلها وإدارتها باستخدام نفس البيئة. إضافة إلى ذلك، فإن سهولة النشر والتنقل تجعل من Node.js خياراً مثالياً، إذ لا يتطلب تشغيل خادم Node.js سوى تثبيت البيئة على الجهاز المستهدف، دون الحاجة إلى إعدادات معقدة أو خوادم ويب إضافية، مما يقلل من احتمالية الثغرات الأمنية الناتجة عن الاعتماديات الخارجية [18]. وقد جرى بناء خادم العارض الرئيسي (الموجود في viewer/server.js) الذي يعمل على المنفذ 8800 باستخدام Node.js، كما جرى بناء خادم اللانشر البديل (الموجود في launcher/server.js) الذي يعمل على المنفذ 8765 باستخدامها أيضاً.



الشكل 1-2 آلية عمل Node.js.

2-2-2 وحدات Node.js المدمجة

اعتمد بناء الخادم في هذا المشروع على وحدات Node.js المدمجة (Built-in Modules) حصراً، دون استخدام أي اعتماديات خارجية (External Dependencies) مثل Express.js أو أي إطار عمل آخر، وهذا القرار يعكس التزام المشروع بمبدأ "صفر اعتماديات" الذي يضمن سهولة نقل المشروع وتشغيله على أي جهاز دون الحاجة إلى تثبيت حزم إضافية أو التعامل مع تعارضات الإصدارات، مما يجعله مثالياً للبيئات التعليمية والبحثية، كما يقلل من سطح الهجوم (Attack Surface) ويحد من مخاطر الثغرات الأمنية الناتجة عن اعتماديات خارجية غير موثوقة [18]. وقد تم استخدام الوحدات المدمجة التالية في بناء الخادم، حيث توفر كل منها وظيفة محددة تسهم في عمل المنصة بشكل متكامل:

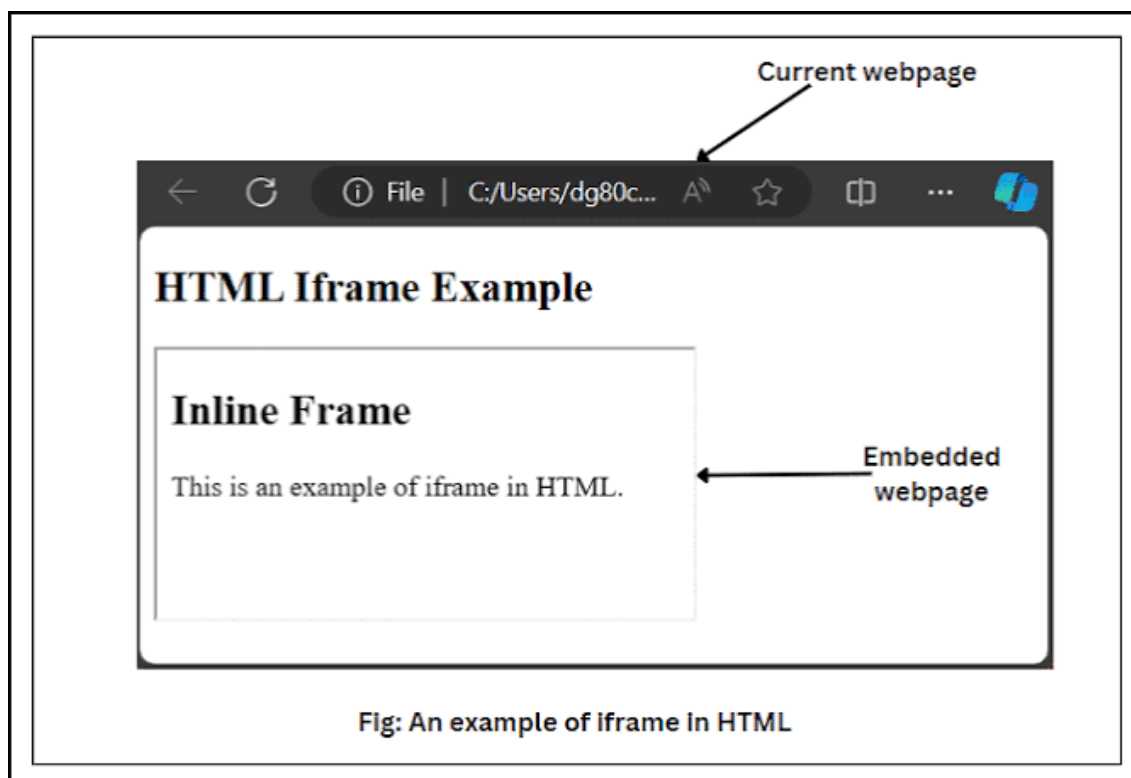
تُستخدم وحدة http لإنشاء خادم HTTP يستمع للطلبات الواردة من المتصفح، ويعالجها، ويرسل الردود المناسبة [19]. أما وحدة fs و fs.promises فتتيح قراءة وكتابة الملفات والمجلدات على القرص الصلب، مثل فحص محتويات مجلد /models، وقراءة ملفات package.json و README.md، وخدمة الملفات الثابتة [20]. وتُستخدم وحدة path لمعالجة مسارات الملفات والمجلدات بطريقة آمنة ومتوافقة مع أنظمة التشغيل المختلفة (Windows)، Linux، macOS [21]. أما وحدة url فتُستخدم لتحليل روابط HTTP واستخراج المعطيات (Query Parameters) منها، مثل استخراج معطيات model و project من طلبات API [22]. وتُعد وحدة child_process من أهم الوحدات في هذا السياق، إذ تُستخدم لتشغيل عمليات خارجية (External Processes) مثل تثبيت الاعتماديات (npm install، pip install) وتشغيل المشاريع (python app.py، node server.js)، وإدارة هذه العمليات عبر دوال spawn و exec [23].

2-3 تقنيات الواجهة الأمامية (Frontend Technologies)

HTML5 1-3-2

تُعد HTML5 (HyperText Markup Language) اللغة الأساسية لإنشاء صفحات الويب وتنظيم محتواها، وهي الإصدار الخامس من معيار HTML الذي يضيف العديد من المزايا الجديدة مثل العناصر الدلالية (Semantic Elements) والدعم المدمج للوسائط المتعددة [24]. وقد جرى استخدام HTML5 لبناء هيكل الصفحة الرئيسية للعارض (viewer/public/index.html) والصفحة الخاصة بلوحة النتائج (benchmarks/index.html). وقد اعتمد التصميم على العناصر الدلالية مثل <header> و <main> و <section> و <footer> لتنظيم المحتوى، مع الاستفادة من عناصر مثل <iframe> لتضمين

المشاريع ومساعد الكود، وعناصر `<button>` و `<div>` لبناء الواجهات التفاعلية التي تتيح للمستخدم التنقل بين المشاريع والنماذج بسهولة.



الشكل 2-2 تقنية iframe في HTML.

CSS3 2-3-2

CSS3 (Cascading Style Sheets) هي لغة تنسيق تستخدم للتحكم في مظهر وتصميم صفحات الويب، وتتيح فصل التنسيق عن المحتوى، مما يسهل صيانة الموقع وتحديثه، ويضيف CSS3 مزايا متقدمة مثل التدرجات اللونية (Gradients)، والظلال (Shadows)، والتحويلات (Transforms)، والرسوم المتحركة (Animations) [25]. جرى استخدام CSS3 في ملف viewer/public/styles.css لتنسيق واجهة العارض، حيث اعتمد التصميم على متغيرات CSS (Custom Properties) لتوحيد الألوان والأحجام عبر الصفحة، وشبكات CSS (CSS Grid) لتوزيع بطاقات المشاريع بشكل متجاوب باستخدام `grid-template-columns: repeat(auto-fill, minmax(280px, 1fr))`، ومرونة

الصناديق (Flexbox) لتنظيم عناصر الرأس (Topbar) والنوافذ المنبثقة (Popups). كما أضيف تأثير الزجاج (Glassmorphism) باستخدام backdrop-filter: blur () لإضفاء مظهر عصري على النوافذ المنبثقة، والرسوم المتحركة (Keyframes) لإنشاء تأثيرات مثل دوران مؤشر التحميل (Spinner) وظهور النوافذ بتأثير انبثاقي، إلى جانب استعلامات الوسائط (Media Queries) لجعل التصميم متجاوباً مع أحجام الشاشات المختلفة، خاصة عند عرض 780px فأقل.

JavaScript (Vanilla JS) 3-3-2

JavaScript هي لغة برمجة عالية المستوى، تُستخدم بشكل رئيسي لإضافة التفاعلية والسلوك الديناميكي إلى صفحات الويب، وقد تم اعتماد JavaScript الخام (Vanilla JS) دون استخدام أي إطار عمل (Framework) أو مكتبة خارجية، وذلك لضمان التوافق مع مبدأ "صفر اعتماديات" الذي يسرع تحميل الصفحة ويبسط عملية التطوير، ويمنح المطور تحكماً كاملاً في سلوك الواجهة دون قيود أو افتراضات تفرضها الأطر العمل، كما يقلل من حجم الملفات المنقولة إلى المتصفح مما يحسن أداء التحميل [26].

جرى استخدام JavaScript في ملف viewer/public/app.js (494 سطراً) لتطوير منطق الواجهة بالكامل، حيث تم استخدام fetch () لاستدعاء نقاط API من الخادم (/api/scan/، /api/launch/، /api/probe/، وغيرها)، وبناء عناصر DOM ديناميكياً عبر دالة مساعدة el(tag, props, ...children) لإنشاء عناصر HTML برمجياً وعرضها في الصفحة، وإدارة الحالة (State Management) عبر متغيرات عامة مثل MODELS و CONFIG و currentProject لتتبع حالة التطبيق. كما أضيفت مستمعات الأحداث (Event Listeners) للتعامل مع نقرات الأزرار، وإغلاق النوافذ، ونسخ النصوص إلى الحافظة باستخدام Clipboard API، والاستقصاء (Polling) لفحص جاهزية الخوادم المشغلة للمشاريع عبر استدعاء /api/probe/ بشكل دوري، وتضمين iframe لتحميل المشاريع ومساعد الكود داخل النوافذ المنبثقة.

2-4 تقنيات المشاريع المولدة (Generated Projects Technologies)

أنتجت النماذج الثلاثة (GLM-5.1، DeepSeek V4 Pro، MiMo V2.5 Pro) مشاريع بتنوع تقني كبير، يعكس قدرات كل نموذج في التعامل مع مختلف أنواع التطبيقات، ويمكن تصنيف هذه المشاريع حسب التقنيات المستخدمة على النحو التالي:

2-4-1 مشاريع Node.js

استُخدمت Node.js مع إطار العمل Express.js في بناء 10 مشاريع (3 لكل من GLM-5.1 و DeepSeek V4 Pro، و MiMo V2.5 Pro)، وتتميز هذه المشاريع بوجود ملف package.json الذي يحدد الاعتماديات، وملف دخول (Entry Point) مثل server.js أو app.js. وقد اشتملت هذه المشاريع على مكتبات شائعة مثل Express.js لإطار العمل، و body-parser لتحليل بيانات الطلبات بتنسيق JSON، و lowdb أو nedb كقواعد بيانات بسيطة تعتمد على ملفات JSON، و ejs أو pug كمحرركات قوالب لإنشاء صفحات HTML ديناميكياً، و cors لإدارة سياسات المشاركة بين المصادر (CORS)، و axios أو node-fetch لإجراء طلبات HTTP إلى واجهات برمجة تطبيقات خارجية. يتم اكتشاف هذه المشاريع تلقائياً بواسطة خادم العارض عبر فحص وجود ملف package.json، ثم تشغيلها باستخدام npm install لتثبيت الاعتماديات، ثم <node <entry لبدء الخادم.

2-4-2 مشاريع Python

استُخدمت Python مع إطار العمل Flask في بناء 6 مشاريع (2 لكل نموذج)، وتشمل المشاريع رقم 11 (منظم ملفات) و 12 (واجهة API مع لوحة بيانات)، وتتميز هذه المشاريع بوجود ملف requirements.txt أو pyproject.toml الذي يحدد الاعتماديات. وقد اشتملت هذه المشاريع على مكتبات مثل Flask كإطار عمل خفيف لبناء تطبيقات الويب وواجهات API، و requests لإجراء طلبات

HTTP، وpython-dotenv لتحميل متغيرات البيئة من ملف .env، وos وshutil للتعامل مع نظام الملفات (لتنظيم الملفات في المشروع 11)، وsqlite3 للتعامل مع قواعد بيانات SQLite. يتم اكتشاف هذه المشاريع عبر فحص وجود ملف requirements.txt، ثم تشغيلها باستخدام `pip install -r requirements.txt` لتثبيت الاعتماديات، ثم `python <entry>` حيث يكون ملف الدخول عادة `app.py` أو `.server.py`.

2-4-3 المشاريع الثابتة (Static Projects)

تشكل المشاريع الثابتة (HTML/CSS/JS) الغالبية العظمى من المخرجات، بواقع 40 مشروعاً من أصل 60، ولا تحتاج هذه المشاريع إلى خادم لتشغيلها، بل يتم عرضها مباشرة عبر متصفح الويب، وتشمل تطبيقات مثل محرر ماركداون، وألعاب الاختبارات، والحاسبات العلمية، وصفحات الهبوط، ومعارض الأعمال، وتطبيقات التصور البياني، والنماذج ثلاثية الأبعاد باستخدام Three.js. وقد استخدمت هذه المشاريع مكتبات عبر CDN مثل Three.js لإنشاء وعرض رسومات ثلاثية الأبعاد في المتصفح (المشاريع 16 و 17)، وChart.js أو D3.js لإنشاء رسوم بيانية وتصورات بيانات تفاعلية، وmarked.js أو markdown-it لتحويل نصوص Markdown إلى HTML (المشروع 06). يتم اكتشاف هذه المشاريع عند عدم وجود package.json أو requirements.txt أو pubspec.yaml، ويتم اعتبارها مشاريع ثابتة، ثم عرضها مباشرة عبر مسار `/project/<model>/` الذي يخدم الملفات الثابتة من مجلد المشروع.

2-4-4 مشاريع Flutter

أنتج نموذجاً GLM-5.1 و MiMo V2.5 Pro مشروعين Flutter حقيقيين لكل منهما (المشاريع 18 و 19)، بينما أنتج DeepSeek V4 Pro صفحات HTML بدلاً من Flutter، وتتميز مشاريع Flutter

بوجود ملف pubspec.yaml ومجلد lib/ يحتوي على ملف main.dart، وتستخدم Flutter SDK كإطار عمل لتطوير تطبيقات متعددة المنصات (Android، iOS، Web، Desktop) من قاعدة كود واحدة، مع لغة Dart كلغة البرمجة الأساسية. يتم اكتشاف هذه المشاريع عبر فحص وجود ملف pubspec.yaml، لكن خادم العارض لا يقوم بتشغيلها تلقائياً بسبب تعقيد بيئة Flutter وحاجة التشغيل إلى أدوات إضافية، وبدلاً من ذلك، يعرض العارض رسالة توجه المستخدم لتشغيل المشروع يدوياً باستخدام flutter pub get و flutter run.

2-5 أدوات التشغيل والنشر (Deployment and Launch Tools)

2-5-1 Batch Scripts (ملفات .bat)

استخدمت لغة البرمجة النصية (Windows Batch Scripting) Batch لإنشاء ملفات التشغيل والإيقاف الموحدة التي تسهل إغلاق وإيقاف مكونات المنصة بنقرة واحدة، وتُعد Batch لغة برمجة نصية بسيطة تُستخدم في نظام Windows لأتمتة المهام عبر سطر الأوامر [27]. تم تطوير المشغل الموحد (Desktop\start.bat) الذي يقوم بالتحقق من توفر node و opencode في PATH، وتحرير المنافذ 8800 و 8801 إذا كانت مشغولة باستخدام netstat و taskkill، وتشغيل خادم العارض في نافذة مصغرة (start /min) على المنفذ 8800، وتشغيل مساعد EHCode (opencode web --port 8801) في نافذة مصغرة، ثم انتظار 4 ثوانٍ وفتح المتصفح على http://localhost:8800. كما تم تطوير الموقف الموحد (Desktop\stop.bat) الذي يقوم بقتل العمليات على المنافذ 8800 و 8801 باستخدام taskkill، وقتل أي عمليات opencode.exe متبقية. إضافة إلى ذلك، يوجد مشغل العارض فقط (banchmarks\start.bat) ومشغل اللانشر البديل (banchmarks\launcher\start-) (launcher.bat) على المنفذ 8765. وقد استخدمت هذه الملفات أوامر Batch الأساسية مثل @ echo

off لإخفاء الأوامر أثناء التنفيذ، و chcp 65001 لضبط ترميز UTF-8 لدعم الأحرف العربية، و where للتحقق من وجود أمر في PATH، و %ERRORLEVEL% لقراءة رمز الخروج من الأمر السابق، و %USERPROFILE% كمتغير بيئي يشير إلى مجلد المستخدم، و 5=tokens "for /f" لاستخراج PID من إخراج netstat، و findstr للبحث عن نصوص محددة في الإخراج، و taskkill /F /PID لقتل عملية بالقوة باستخدام معرفها، و taskkill /F /IM لقتل عملية بالاسم، و start /min لتشغيل برنامج في نافذة مصغرة، و "URL" "start" لفتح رابط في المتصفح الافتراضي، و N /t timeout للانتظار لعدد N من الثواني.

opencode / EHCode 2-5-2

opencode هو وكيل برمجة بالذكاء الاصطناعي مفتوح المصدر، يعمل كأداة تفاعلية تساعد المطورين على كتابة الكود في الطرفية (Terminal) أو بيئة التطوير المتكاملة (IDE)، ويدعم أكثر من 75 مزوداً لنماذج الذكاء الاصطناعي، بما في ذلك النماذج السحابية (Claude، GPT، Gemini) والنماذج المحلية (Ollama) [28]. جرى استخدام opencode web --port 8801 لتشغيل خادم ويب لمساعد الكود، ويتم تضمينه داخل iframe في العارض لتوفير واجهة دردشة تفاعلية، كما استخدم opencode serve في بعض السيناريوهات لتشغيل خادم API صامت (Headless)، و opencode run "... لتشغيل أوامر برمجية محددة. وقد تم توفير توزيعية محمولة (dist-ehcode) تحتوي على نسخة مجمعة من opencode على شكل ehcode.exe مستقل بالكامل، لا يحتاج إلى Node.js أو npm، ويمكن تشغيله من أي مجلد، وتأتي مع install.bat لتنصيب ehcode.exe في PATH وتفعيل Playwright MCP، و uninstall.bat لإزالة التنصيب.

2-6 تقنيات الذكاء الاصطناعي والمساعدة البرمجية (AI and Coding Assistant Technologies)

2-6-1 opencode (وكيل البرمجة بالذكاء الاصطناعي)

كما ذكر في القسم السابق، يُستخدم opencode كمساعد برمجي ذكي مدمج داخل المنصة، ويعتمد على نموذج لغوي كبير (LLM) لتوليد الردود والاقتراحات البرمجية، ويدعم مجموعة واسعة من النماذج من مختلف المزودين [28]. وتشمل الميزات الرئيسية المستخدمة في المشروع واجهة الويب التفاعلية التي توفر نافذة دردشة مدمجة داخل العارض عبر iframe، ودعم النماذج المتعددة لإتاحة اختيار النموذج المناسب (Claude، GPT، Gemini، إلخ)، وإدارة الجلسات للحفاظ على سياق المحادثة عبر الجلسات، وتكامل MCP لدعم بروتوكول Model Context Protocol لتوسيع القدرات.

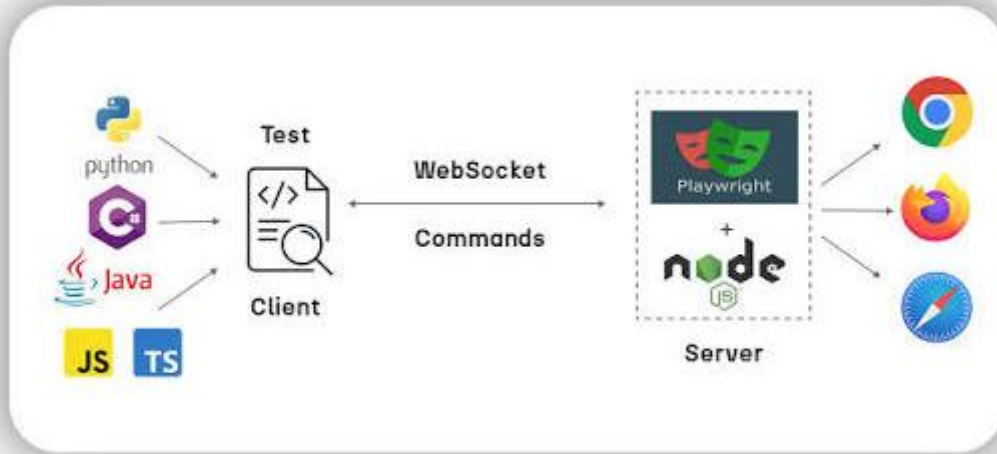
2-6-2 بروتوكول (MCP) Model Context Protocol

MCP (Model Context Protocol) هو بروتوكول مفتوح المصدر يهدف إلى توحيد طريقة تواصل تطبيقات الذكاء الاصطناعي مع مصادر البيانات والأدوات الخارجية، ويوفر طريقة موحدة لربط النماذج اللغوية بأنظمة الملفات، وقواعد البيانات، والمتصفحات، وأدوات التطوير، مما يعزز قدرة النماذج على أداء مهام معقدة [29]. جرى تفعيل Playwright MCP في install.bat الخاص بتوزيع EHCode المحمولة، مما يسمح لمساعد الكود بالتحكم في متصفح حقيقي (عبر Playwright) لأتمتة الاختبارات والتفاعل مع صفحات الويب.

7-2 أدوات التقييم والمراقبة (Evaluation and Monitoring Tools)

Playwright 1-7-2

Playwright هو إطار عمل مفتوح المصدر لأتمتة المتصفحات، ويدعم اختبار تطبيقات الويب عبر متصفحات Chrome و Firefox و Safari و Edge، ويوفر واجهة برمجة تطبيقات موحدة للتفاعل مع العناصر، وإجراء النقرات، وملء النماذج، والتقاط لقطات الشاشة [30]. يستخدم Playwright عبر MCP (كما ذكر أعلاه) لأتمتة اختبار المشاريع المولدة في متصفح حقيقي، مما يمكن من التحقق من قابلية التشغيل الفعلية وسلوك التطبيقات، وتوجد مجلدات مثل `playwright-mcp` تحتوي على سجلات ولقطات من عمليات الاختبار الآلي.



الشكل 2-3 آلية عمل Playwright.

Sisyphus 2-7-2

Sisyphus هو نظام لإدارة استمرارية الجلسات (Session Continuation) في تطبيقات الذكاء الاصطناعي، يسمح بحفظ واستئناف جلسات العمل، ويُستخدم لتخزين نقاط التوقف (Checkpoints)

لعمليات توليد الكود الطويلة أو المعقدة. يوجد مجلد `/sisyphus/run-continuation/` يحتوي على نقاط حفظ لجلسات الذكاء الاصطناعي، مما يسمح باستئناف العمل من حيث توقف دون فقدان السياق.

الفصل الثالث: تحليل وتصميم النظام

3-1 تمهيد

يمثل تحليل النظام وتصميمه المرحلة المحورية التي تنتقل فيها المتطلبات المجردة إلى نموذج معماري ملموس قابل للتنفيذ. يعتمد هذا الفصل على منهجية النمذجة البصرية باستخدام لغة النمذجة الموحدة (UML) لوصف هيكلية النظام وسلوكه الديناميكي، مع التركيز على تحقيق أهداف المشروع المتمثلة في توفير منصة تقييم مرنة، قابلة للنقل، وتفاعلية، تجمع بين عرض المشاريع المولدة، وتشغيلها التلقائي، والمساعدة البرمجية الذكية في بيئة موحدة. يتضمن الفصل تحليلاً للمتطلبات الوظيفية وغير الوظيفية، وتصميماً لهيكلية النظام ومكوناته، وتصميماً لحالات الاستخدام التي تعكس تفاعل المستخدمين مع المنصة، وتصميماً ديناميكياً يوضح تسلسل العمليات وآليات التفاعل بين الخادم والواجهة الأمامية، وأخيراً تصميم إدارة الحالة والعمليات الذي يضمن الاستقرار والكفاءة أثناء تنفيذ المهام البرمجية المتعددة.

3-2 تحليل المتطلبات

3-2-1 المتطلبات الوظيفية

تُعرّف المتطلبات الوظيفية على أنها الخدمات التي يجب على النظام تقديمها للمستخدمين، وتشمل العمليات والتفاعلات الملموسة التي تدعم سير العمل الأساسي للمنصة. تم استخلاص المتطلبات الوظيفية التالية:

- فحص المجلدات تلقائياً: يجب على النظام فحص مجلد `models` تلقائياً للكشف عن النماذج المتاحة والمشاريع المولدة، وتصنيف كل مشروع حسب التقنية المستخدمة (Node.js، Python، Static، Flutter) بناءً على وجود ملفات محددة مثل `package.json` أو `requirements.txt` أو `pubspec.yaml`.

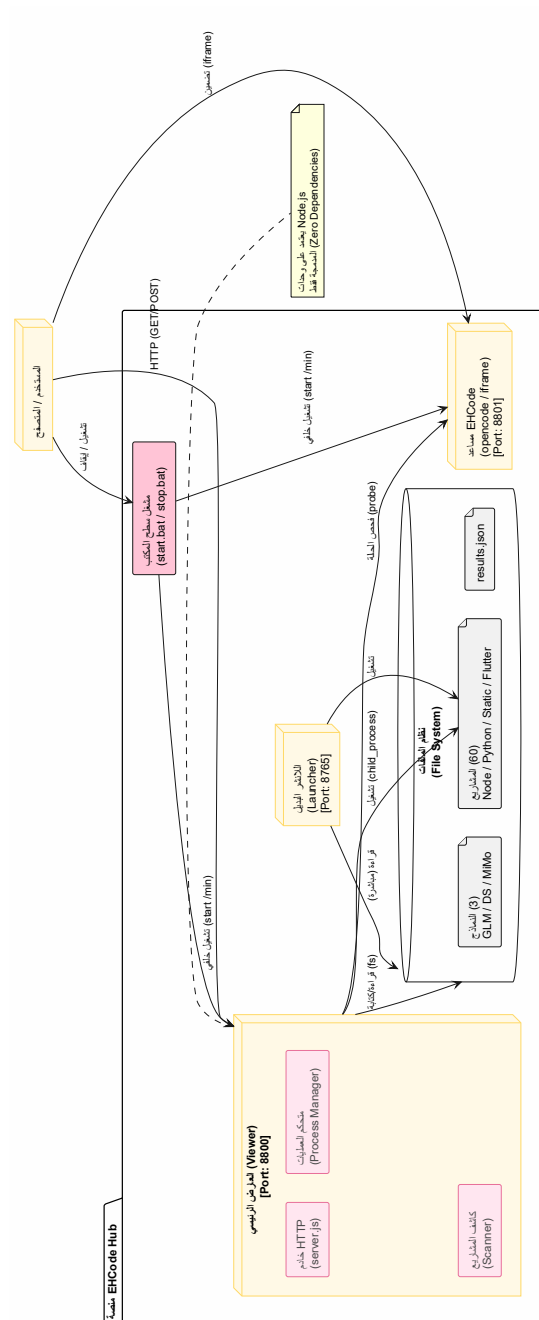
- عرض المشاريع في واجهة تفاعلية: يجب عرض المشاريع في واجهة مستخدم على شكل بطاقات قابلة للطي (Accordion) مصنفة حسب النموذج المُنتج، مع إظهار معلومات أساسية عن كل مشروع كالنوع والمنفذ ونقطة الدخول.
- تشغيل المشاريع تلقائياً: يجب أن يتمكن المستخدم من تشغيل أي مشروع بنقرة زر، مع قيام النظام تلقائياً بتنصيب الاعتماديات المطلوبة (pip install، npm install) وإطلاق الخادم الخلفي في الخلفية، مع إدارة المنافذ وتحريرها بشكل آلي.
- عرض المشاريع المشغلة: يجب عرض المشاريع المشغلة داخل نافذة iframe مباشرة في المتصفح، مع دعم أوضاع التشغيل المختلفة (Static، Node.js، Python) وحالة الاستثناء لمشاريع Flutter التي تتطلب تشغيلاً يدوياً.
- توليد دليل توثيقي تفاعلي: يجب على النظام توليد دليل لكل مشروع بشكل تلقائي، يتضمن المتطلبات المسبقة، أوامر التنصيب والتشغيل، أوامر التحقق، النتائج المتوقعة، قائمة الاعتماديات، وقائمة الملفات.
- مقارنة النماذج: يجب توفير جدول مقارنة بين النماذج الثلاثة يعرض مؤشرات الأداء الرئيسية كعدد المشاريع، وتنوع الأنواع التقنية، وجودة التوثيق، والالتزام بالموصفات.
- دمج مساعد برمجي ذكي: يجب دمج مساعد كود (EHCode/opencode) داخل واجهة العارض عبر iframe، مع إمكانية فتحه بنقرة زر والتفاعل معه دون مغادرة المنصة.
- إدارة العمليات المشغلة: يجب على النظام إدارة العمليات المشغلة من بدء وإيقاف وتحرير تلقائي للمنافذ، مع تتبع الحالة وتسجيل الأخطاء والمخرجات لتسهيل التشخيص.
- توفير مشغل موحد: يجب توفير مشغل موحد (start.bat) يقلع العارض والمساعد معاً بنقرة واحدة، ومشغل إيقاف (stop.bat) لتنظيف الموارد وإعادة ضبط المنافذ.

3-2-2 المتطلبات غير الوظيفية

تحدد المتطلبات غير الوظيفية معايير الأداء والجودة والقيود التي يجب أن يلتزم بها النظام أثناء تقديمه للخدمات الوظيفية، وتشمل:

- قابلية النقل: يجب أن يعمل النظام على أي جهاز مثبت عليه Node.js، دون حاجة إلى تثبيت اعتماديات إضافية، وذلك بفضل الاعتماد على الوحدات المدمجة فقط.
- الأداء: يجب ألا تتجاوز مدة فحص المشاريع الستين بضع ثوانٍ، مع قدرة خادم العارض على التعامل مع عدة طلبات متزامنة دون تباطؤ كبير، مستفيداً من نموذج الإدخال/الإخراج غير المحظور.
- الأمان: منع هجمات عبور المسار (Path Traversal) عبر التحقق من صحة المسارات، وتقليل سطح الهجوم (Attack Surface) عبر تجنب الاعتماديات الخارجية غير الموثوقة.
- سهولة الاستخدام: توفير واجهة مستخدم عربية بديهية، مع إرشادات واضحة، وأزرار تفاعلية، وتغذية راجعة فورية للمستخدم أثناء تشغيل المشاريع وعمليات التحميل.
- التوافق: دعم أنظمة التشغيل Windows مع إمكانية التوسع لأنظمة Unix/Linux لاحقاً، ودعم بيئات التطوير المختلفة (Flutter، Python، Node.js) مع توفير رسائل خطأ واضحة في حال نقص الأدوات المطلوبة.

3-3 تصميم هيكلية النظام



الشكل 1-3 مخطط هيكلية العمل.

يعتمد التصميم المعماري للمنصة على الفصل بين المكونات الرئيسية ووضوح المسؤوليات، مع تحديد آليات الاتصال بينها. يتكون النظام من أربع كتل رئيسية كما هو موضح في مخطط الهيكلية في الشكل (1-3) خادم العارض الرئيسي (Viewer) الذي يمثل بوابة النظام، ومساعد EHCode (opcode)

الذي يعمل كخدمة مستقلة، واللائشر البديل (Launcher) كخيار إضافي للتشغيل السريع، ونظام الملفات (File System) الذي يشكل طبقة البيانات لتخزين المشاريع والنتائج.

يتفاعل المتصفح مع العارض عبر بروتوكول HTTP، حيث يستقبل العارض طلبات المستخدم ويعالجها. يضمّن العارض مساعد EHCode عبر iframe، مما يتيح للمستخدم الاستفادة من المساعد دون مغادرة المنصة. يتواصل العارض مع نظام الملفات عبر وحدات fs و path لقراءة المشاريع وكتابة البيانات، ومع العمليات الخارجية عبر child_process لإطلاق المشاريع وإدارتها.

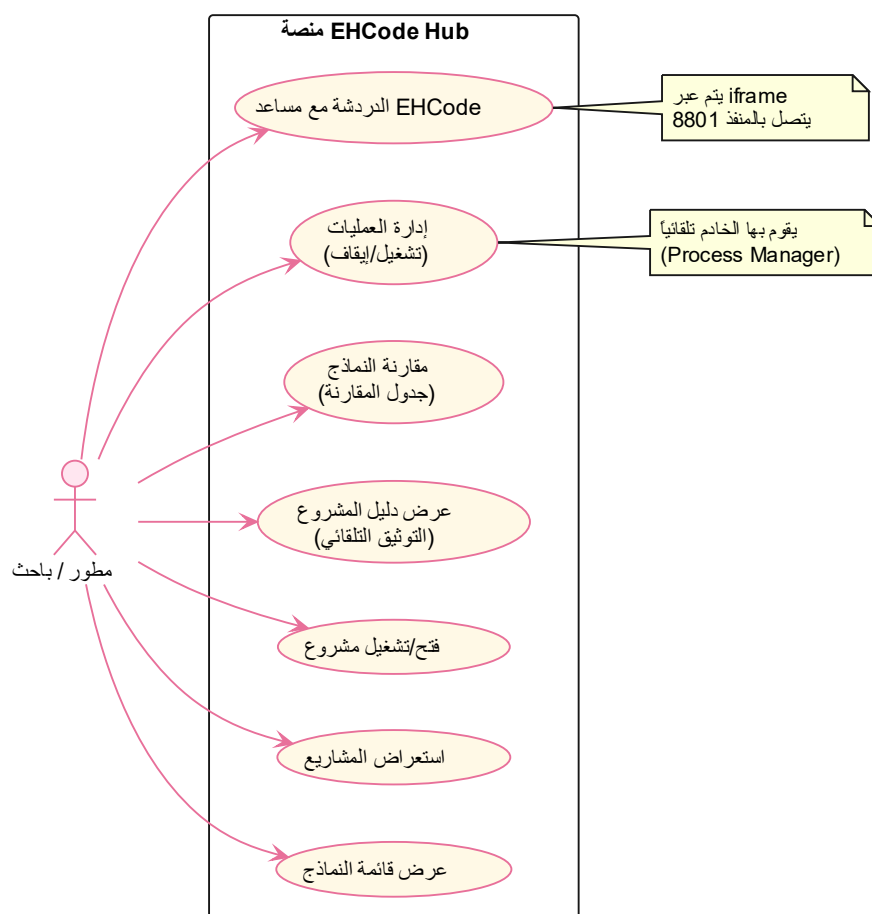
يُعد المشغل الموحد (start.bat) المكون الذي يربط الكتل معاً في بيئة التشغيل، حيث يقلع العارض على المنفذ 8800 ومساعد EHCode على المنفذ 8801 في الخلفية، ثم يفتح المتصفح على الواجهة الرئيسية. يجسد هذا التصميم مبدأ "الفصل بين الاهتمامات" (Separation of Concerns)، حيث تُفرد كل وحدة بمهمة محددة، مما يُسهل الصيانة والتطوير المستقبلي، ويضمن عدم تأثر المكونات ببعضها البعض في حال حدوث أعطال.

3-4 تصميم حالات الاستخدام

يُمثل مخطط حالات الاستخدام جميع التفاعلات الممكنة بين الممثل الرئيسي (المستخدم/الباحث) والنظام، ويحدد نطاق الخدمات التي تقدمها المنصة. تم تحديد سبع حالات استخدام رئيسية:

- أولاً: عرض قائمة النماذج، حيث يقوم النظام بفحص مجلد models/ وعرض النماذج الثلاثة كبطاقات قابلة للطّي، تعرض كل بطاقة اسم النموذج والمزود وعدد المشاريع المنتجة.
- ثانياً: استعراض المشاريع، حيث يتم عرض قائمة المشاريع لكل نموذج مع معلومات موجزة عن كل مشروع تشمل رقمه وعنوانه ونوعه التقني (Node.js، Python، Static، Flutter) والمنفذ المُكتشف ونقطة الدخول.

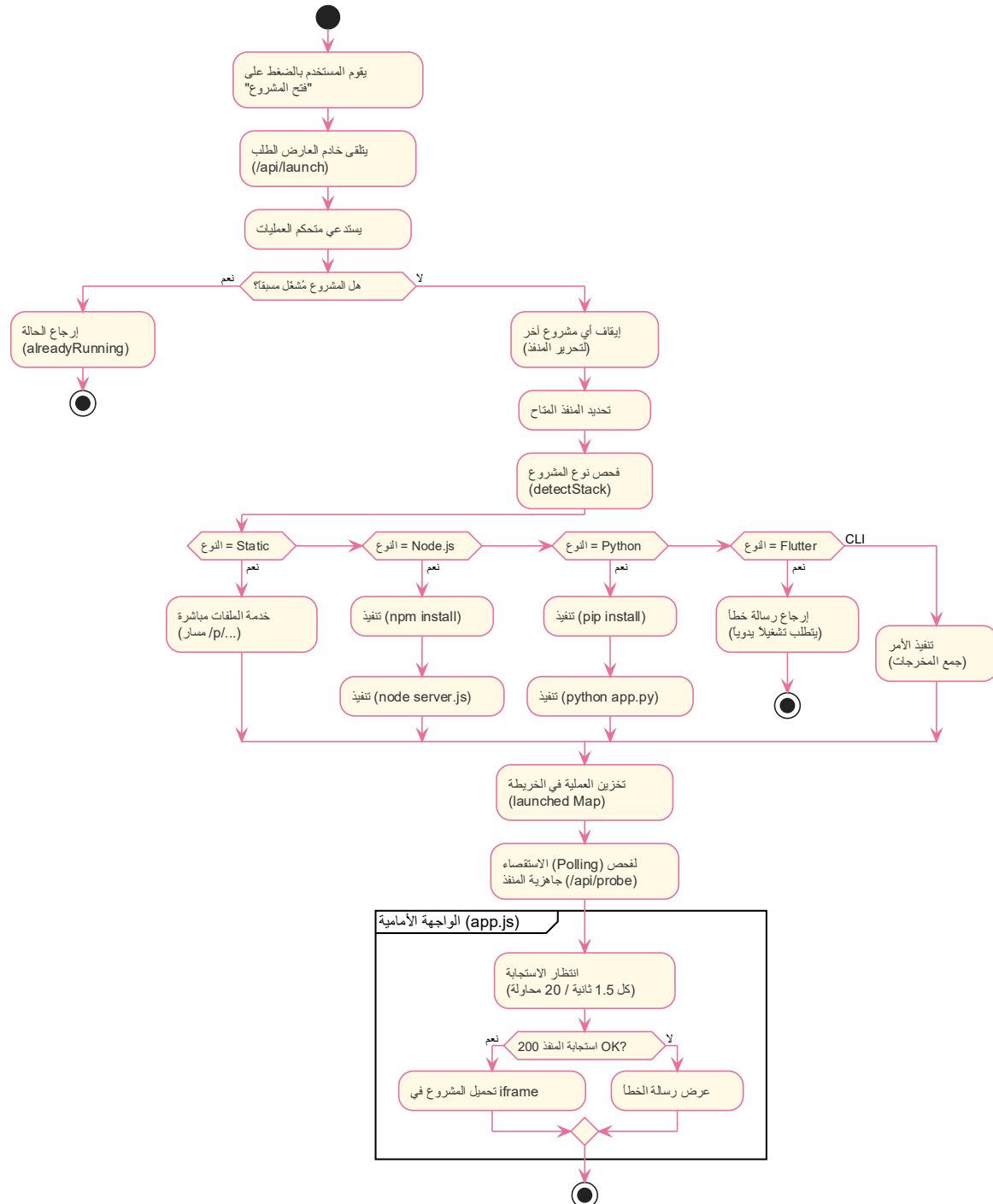
- ثالثاً: فتح وتشغيل مشروع، وهي حالة الاستخدام الأكثر تعقيداً، حيث يتم تشغيل مشروع محدد تلقائياً عند الطلب، مع تثبيت الاعتماديات وإطلاق الخادم وعرض المشروع داخل نافذة iframe، وتتم إدارة المنافذ والعمليات بشكل آلي بالكامل.
- رابعاً: عرض دليل المشروع، حيث يقوم النظام بتوليد وعرض دليل تفاعلي لكل مشروع يتضمن المتطلبات المسبقة وأوامر التثبيت والتشغيل والتحقق والنتائج المتوقعة والاعتماديات وقائمة الملفات.
- خامساً: مقارنة النماذج، حيث يتم عرض جدول مقارنة يلخص أداء النماذج بناءً على عدد المشاريع، وجودة التوثيق، والتنوع التقني، والالتزام بالمواصفات، مع تقديم توصية تلقائية بالنموذج الأنسب.
- سادساً: الدردشة مع مساعد EHCode، حيث يتم فتح نافذة iframe متصلة بمساعد الكود لطرح الأسئلة البرمجية والحصول على اقتراحات وتوضيحات فورية.
- سابعاً: إدارة العمليات، وهي حالة استخدام خلفية يقوم بها النظام تلقائياً، وتشمل بدء العمليات وإيقافها، وتحرير المنافذ، وتتبع الحالة، وتسجيل الأخطاء، وإعادة المحاولة في حال الانهيار.
- يتكامل الممثل "مستخدم" مع جميع حالات الاستخدام من خلال الواجهة الرسومية، بينما تعمل حالات الاستخدام المتعلقة بالعمليات بشكل وثيق مع حالة استخدام تشغيل المشروع لضمان تجربة سلسلة وخالية من التعقيدات التقنية.



الشكل 2-3 مصمم حالات الاستخدام.

3-5 تصميم التفاعلات الديناميكية

3-5-1 مخطط النشاط - تشغيل مشروع



الشكل 3-3 مخطط النشاط.

يعكس مخطط النشاط (Activity Diagram) سير العمل الكامل بدءاً من طلب المستخدم تشغيل مشروع معين، مروراً بعمليات الخادم الداخلية، وانتهاءً بعرض النتيجة في المتصفح. يبدأ التدفق عند ضغط المستخدم على زر "فتح المشروع" في بطاقة المشروع. يستقبل خادم العارض الطلب عبر نقطة النهاية `api/launch/`، ثم يستدعي متحكم العمليات (Process Manager).

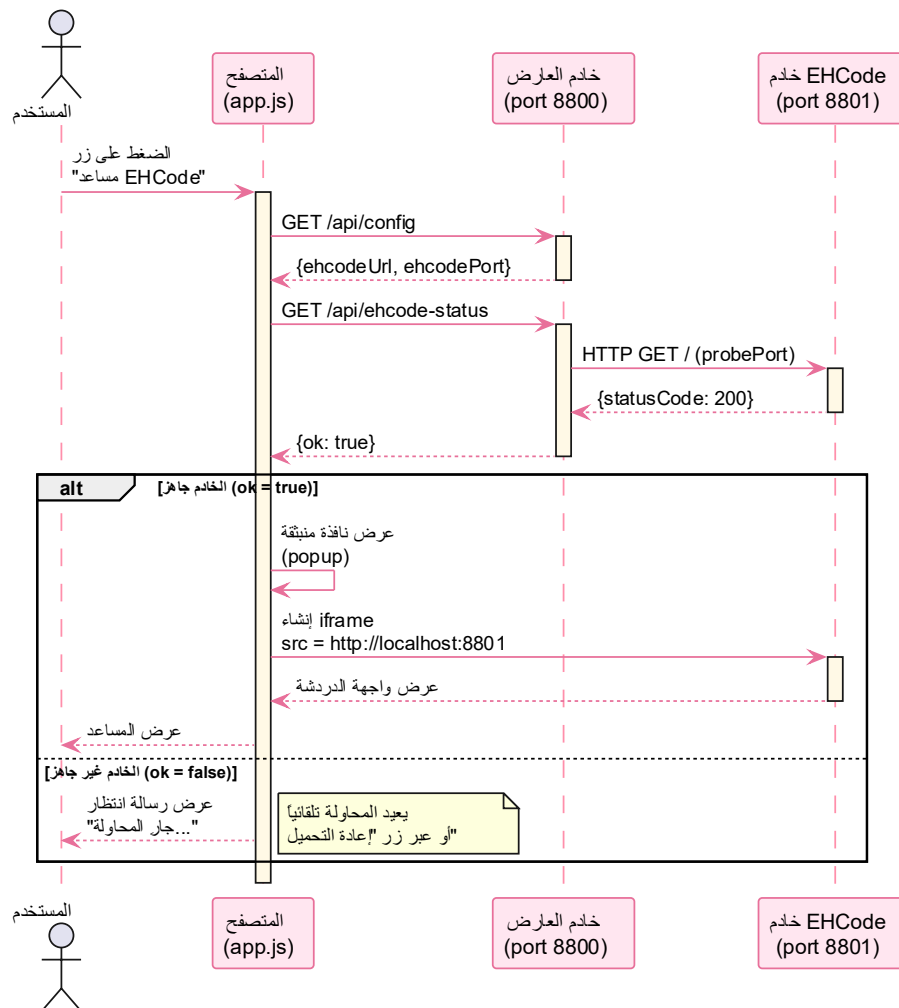
يقوم متحكم العمليات أولاً بالتحقق مما إذا كان المشروع مفتوحاً مسبقاً، وفي حال عدم تشغيله يُقدم على إيقاف أي مشروع آخر لتحرير المنفذ، مما يضمن عدم تعارض المنافذ. بعد ذلك يُحدد المنفذ المتاح ويُصنف نوع المشروع عبر دالة الكشف (`detectStack`) وفق الترتيب التالي: إن كان المشروع من النوع الثابت (Static) يُعالج مباشرة كملف ثابت ويُخدم عبر المسار `/project/<model>/`، وإن كان من نوع Node.js يُنفذ `npm install` (إن وجد `package.json`) ثم يُشغل `node <entry>`، وإن كان من نوع Python يُنفذ `pip install` (إن وجد `requirements.txt`) ثم يُشغل `python <entry>`، وأما إن كان من نوع Flutter فيُرجع رسالة خطأ توجه المستخدم للتشغيل اليدوي نظراً لتعقيد بيئته.

بعد نجاح spawn العملية، يُخزن مرجعها في خريطة العمليات (launched Map) ويرسل `pid` و `port` للواجهة. تشرع الواجهة الأمامية فوراً في حلقة استقصاء (Polling) باستدعاء `api/probe?port=<port>` كل 1.5 ثانية. يستمر الاستقصاء حتى تستجيب العملية بـ HTTP 200 فتُحمّل في `iframe`، أو تتجاوز عدد المحاولات (20 محاولة) فتُعرض رسالة خطأ توضح سبب الفشل، مع إمكانية إعادة المحاولة يدوياً.

3-5-2 مخطط التسلسل - فتح مساعد EHCode

يوضح مخطط التسلسل (Sequence Diagram) الموضح في الملحق (الشكل 3-4) تبادل الرسائل بين المكونات الثلاثة (المتصفح، خادم العارض، خادم EHCode) أثناء محاولة المستخدم فتح المساعد الذكي.

يبدأ التفاعل بضغط المستخدم على زر "مساعد EHCode" في أعلى الصفحة. يستدعي كود الواجهة الأمامية نقطة `api/config/` للحصول على عنوان ومنفذ خادم EHCode (افتراضياً `http://localhost:8801`). بعد ذلك يطلب المتصفح من العارض فحص حالة المساعد عبر `api/ehcode-status/`، فيقوم العارض (من جانب الخادم) بإجراء فحص المنفذ (`probePort`) على `8801`، وهذا التصميم يتجنب مشاكل CORS التي قد تمنع المتصفح من فحص خادم آخر مباشرة.

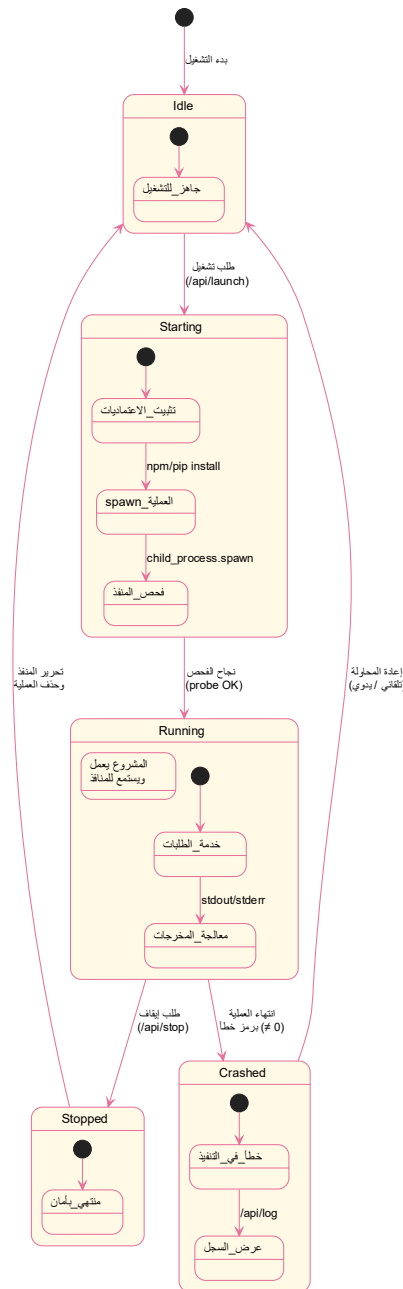


الشكل 3-4 مخطط التسلسل.

في حال كانت الاستجابة `ok: true`، يعرض المتصفح نافذة منبثقة ويضبط `src` الخاص بـ `iframe` على رابط `EHCode`، فُحْمَل واجهة الدردشة تلقائياً وتكون جاهزة للاستخدام الفوري. أما في حال عدم الجاهزية، يعرض المتصفح رسالة انتظار تُعلم المستخدم بأن الخادم لم يستجب بعد، ويُتيح زر إعادة التحميل، مفسحاً مجالاً للمستخدم للمحاولة مجدداً بعد إقلاع الخادم، مما يوفر تجربة مرنة وغير قاطعة.

3-6 تصميم إدارة الحالة والعمليات

يُعد متحكم العمليات (Process Manager) المكون المسؤول عن تتبع دورة حياة كل مشروع مُطَلَق، مما يحافظ على استقرار النظام من خلال إدارة العمليات غير المتزامنة. يُخزَّن كل مشروع في خريطة العمليات (launched Map) مفتاحه هو `modelId/projectId` وقيمته كائن يحوي: مرجع العملية (proc)، والمنفذ المخصص (port)، ومعرف العملية في نظام التشغيل (pid)، والحالة الحالية (status)، وسجل المخرجات (log). يُوضح مخطط الحالة هذه التحولات بالتفصيل:



الشكل 3-5 مخطط الحالة.

يمر كل مشروع بسلسلة من الحالات المتعاقبة: يبدأ المشروع في حالة السكون (Idle) حيث يكون جاهزاً للتشغيل ولكن لم يُطلب تشغيله بعد. عند طلب التشغيل، ينتقل إلى حالة البدء (Starting) حيث يتم تثبيت الاعتماديات وتجهيز البيئة، وفي هذه الأثناء يكون المشروع غير متاح بعد للمستخدم. في حال نجاح عملية spawn وبدء الخادم، ينتقل المشروع إلى حالة التشغيل (Running) حيث يعمل ويستمع للمنافذ ويكون

متاحاً للمستخدم عبر `iframe`. في حال فشل `spawn` أو انتهت العملية برمز خطأ، ينتقل المشروع إلى حالة الانهيار (Crashed) حيث يتم تسجيل الأخطاء وعرضها للمستخدم عبر نقطة `api/log/`. أما في حال طلب المستخدم إيقاف المشروع، فينتقل إلى حالة التوقف (Stopped) حيث تُحرر الموارد ويُطلق المنفذ، ثم يعود تلقائياً إلى حالة السكون. يُلاحظ أن الانتقال من حالة التشغيل إلى حالة الانهيار يُطلق آلية تسجيل الأخطاء لعرض المخرجات القياسية وأخطاء `stderr`، مما يساعد في تشخيص الأعطال، كما يضمن التصميم إصدار المنافذ (`releasePort()`) فور انتهاء العملية بأي شكل، سواء بالطريقة النظامية عبر `taskkill /F /T /PID` لقتل شجرة العمليات على Windows، أو عبر `proc.kill()` كإجراء احتياطي، مما يمنع تسرب الموارد ويحافظ على أداء النظام بمرور الوقت.

الفصل الرابع: تنفيذ النظام والتطبيق

العملي

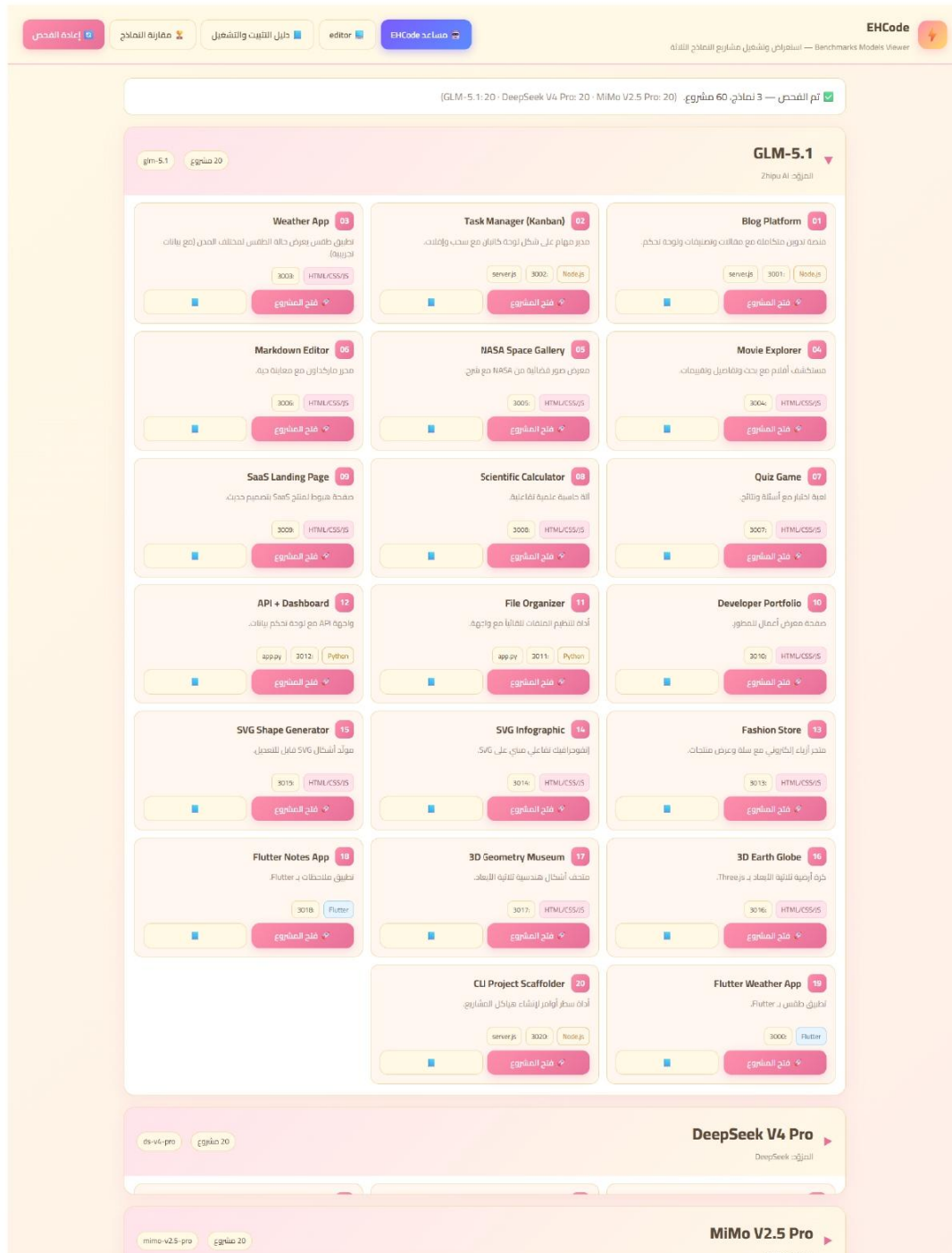
4-1 المقدمة

يتناول هذا الفصل تنفيذ النظام والتطبيق العملي للمنصة، من خلال استعراض أهم الوظائف والميزات التي تم تطويرها لدعم تحليل الشيفرات البرمجية وتحسين جودتها باستخدام تقنيات الذكاء الاصطناعي. وقد روعي في تصميم النظام توفير بيئة عمل متكاملة تُمكن المستخدم من إدارة المشاريع البرمجية، وتحليلها، وتحسينها، وتحويلها بين أطر العمل المختلفة، إضافةً إلى تحرير الشيفرات البرمجية ومقارنتها باستخدام نماذج الذكاء الاصطناعي.

ويستعرض هذا الفصل آلية عمل كل ميزة بصورة مستقلة، مع توضيح الواجهات المرتبطة بها ودورها في تنفيذ المهام المختلفة داخل النظام، مدعومةً بأمثلة تطبيقية توضح كيفية الاستفادة من إمكانيات المنصة في التعامل مع المشاريع البرمجية الحقيقية، وانتهاءً بعرض نتائج مقارنة النماذج المستخدمة في توليد وتحليل الشيفرات البرمجية.

4-2 إدارة المشاريع البرمجية

تُعد إدارة المشاريع البرمجية نقطة الانطلاق الأساسية داخل منصة EHCCode، إذ توفر بيئة موحدة لاستعراض المشاريع البرمجية التي تم توليدها باستخدام نماذج الذكاء الاصطناعي المختلفة، وإدارتها، وتشغيلها، ومتابعة نتائجها. وقد صُممت هذه الواجهة لتسهيل الوصول إلى المشاريع وتنظيمها، بحيث يستطيع المستخدم الانتقال بين النماذج المختلفة، واختيار أي مشروع، ثم تشغيله أو تحسينه أو مقارنته ضمن بيئة عمل واحدة دون الحاجة إلى استخدام أدوات خارجية.



الشكل 1-4 الواجهة الرئيسية لإدارة واستعراض المشاريع البرمجية في منصة EHCCode.

تعرض هذه الواجهة الصفحة الرئيسية لإدارة المشاريع البرمجية داخل منصة EHCCode، والتي تمثل مركز التحكم الرئيس لجميع المشاريع المولدة بواسطة نماذج الذكاء الاصطناعي. وقد روعي في تصميمها تنظيم عناصر الواجهة بصورة تتيح الوصول السريع إلى مختلف وظائف النظام وإدارة المشاريع بكفاءة.

يظهر في أعلى الواجهة شريط التنقل الرئيس الذي يضم مجموعة من الأدوات الأساسية، وتشمل:

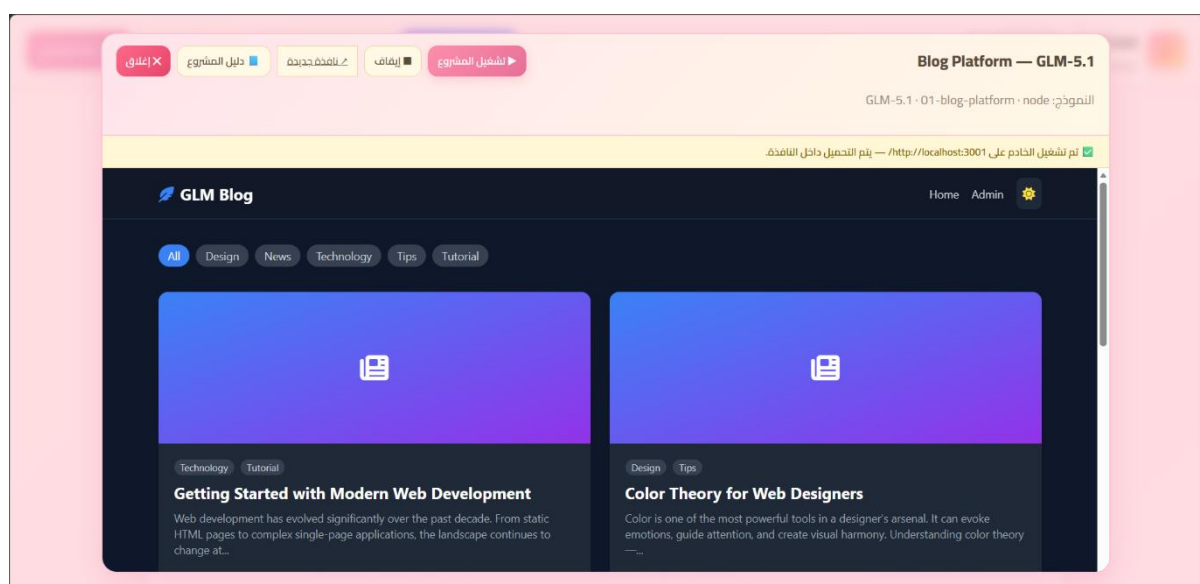
- زر مساعد **EHCode**: يتيح الانتقال إلى المساعد البرمجي الذكي للتفاعل مع نماذج الذكاء الاصطناعي، وطرح الاستفسارات، وتحليل الشيفرات البرمجية، وطلب تحسينها أو تطويرها.
 - زر **Editor**: يفتح محرر الأكواد الذكي الذي يوفر بيئة متكاملة لكتابة الشيفرات البرمجية، وتعديلها، وتحليلها، وتحويلها بين لغات البرمجة المختلفة باستخدام الذكاء الاصطناعي.
 - زر دليل التثبيت والتشغيل: يعرض ملفات وإرشادات تشغيل المشاريع ومتطلباتها البرمجية، مما يساعد المستخدم على تشغيل المشروع في البيئة المناسبة.
 - زر مقارنة النماذج: ينقل المستخدم إلى صفحة مقارنة نماذج الذكاء الاصطناعي، والتي تعرض مؤشرات الأداء ومعايير جودة البرمجيات والنتائج النهائية لكل نموذج.
 - زر إعادة الفحص: يعيد تنفيذ عملية فحص المشاريع المولدة وتحديث بياناتها وإحصاءاتها، بهدف التأكد من سلامة الملفات ونتائج التشغيل بعد أي تعديل أو تحديث.
- أسفل شريط التنقل يظهر شريط الحالة العام، والذي يعرض نتائج عملية الفحص الأخيرة، ويتضمن عدد المشاريع التي تم تحليلها، وعدد النماذج المشاركة في عملية التقييم، مع إظهار حالة اكتمال عملية الفحص، مما يوفر للمستخدم ملخصاً سريعاً عن قاعدة المشاريع الموجودة داخل المنصة.
- بعد ذلك نُقسّم الصفحة إلى أقسام مستقلة، بحيث يمثل كل قسم أحد نماذج الذكاء الاصطناعي المستخدمة في الدراسة، مثل GLM-5.1 و DeepSeek V4 Pro و MiMo V2.5 Pro، مع إظهار اسم النموذج وعدد المشاريع التي تم توليدها باستخدامه، الأمر الذي يسهّل مقارنة المشاريع وتنظيمها وفق النموذج المسؤول عن إنتاجها.

داخل كل قسم تُعرض المشاريع على هيئة بطاقات (Project Cards)، حيث تمثل كل بطاقة مشروعًا مستقلًا، وتتضمن مجموعة من المعلومات الأساسية، من أهمها:

- اسم المشروع.
- وصفًا مختصرًا لوظيفته.
- رقم المشروع ضمن مجموعة النموذج.
- لغة البرمجة أو التقنية المستخدمة في تطويره.
- رقم منفذ التشغيل (Port) الخاص بالمشروع عند الحاجة.
- حالة المشروع وإمكانية تشغيله.

كما تحتوي كل بطاقة على زر "فتح المشروع" الذي يتيح تشغيل المشروع مباشرة والانتقال إلى واجهته التطبيقية، بالإضافة إلى مؤشر يوضح حالة التشغيل، مما يسمح للمستخدم بالوصول السريع إلى المشروع المطلوب دون الحاجة إلى تنفيذ أوامر يدوية.

يساهم هذا التنظيم في تسهيل إدارة عدد كبير من المشاريع البرمجية داخل المنصة، حيث تم اختبار أكثر من عشرين مشروعًا برمجيًا باستخدام كل نموذج من النماذج المدروسة، مع توفير واجهة موحدة تتيح استعراضها والوصول إليها بسرعة، وهو ما يشكل الأساس الذي تُبنى عليه بقية وظائف النظام مثل تحسين المشاريع، وتحويلها، ومقارنتها، وتحليل شيفراتها البرمجية.

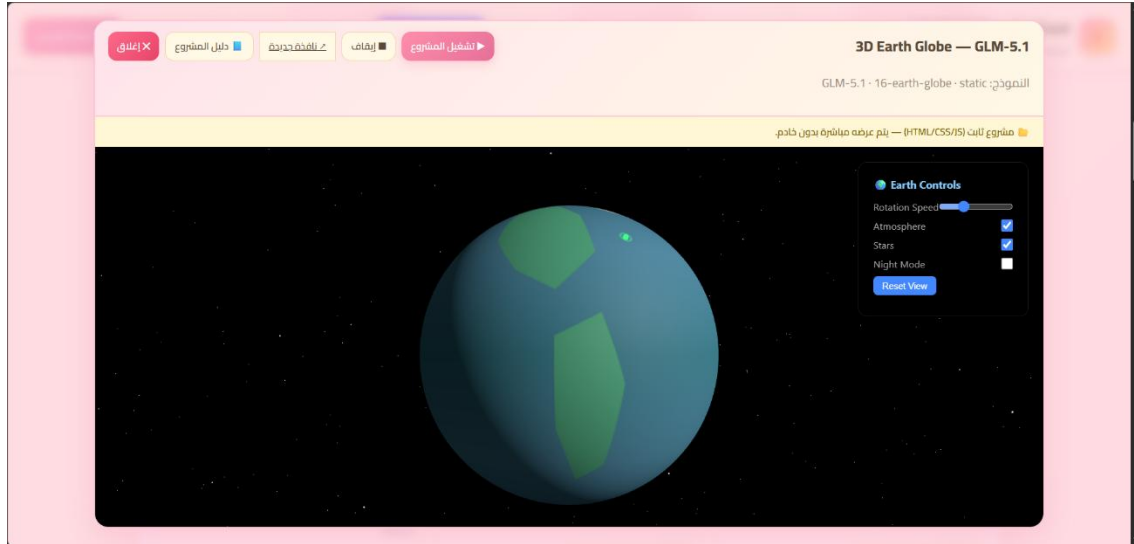


الشكل 2-4: تشغيل مشروع منصة التدوين (Blog Platform) داخل منصة EHCode.

تعرض هذه الواجهة تشغيل مشروع Blog Platform بعد اختياره من قائمة المشاريع في منصة EHCode، حيث يمثل هذا المشروع أحد المشاريع التي تم توليدها باستخدام نموذج GLM-5.1. ويهدف عرض هذا المثال إلى توضيح قدرة النظام على إدارة المشاريع البرمجية وتشغيلها بصورة مباشرة داخل بيئة العمل.

تظهر الواجهة الرئيسية لمنصة التدوين بعد تشغيل المشروع بنجاح، والتي تتضمن العناصر الرئيسية لموقع المدونات، مثل شريط التنقل، وقائمة المقالات، وصور المقالات، والتصنيفات، بالإضافة إلى أدوات التنقل بين أقسام الموقع، مما يدل على سلامة تشغيل المشروع واكتمال مكوناته البرمجية.

ويعكس هذا المثال قدرة النظام على الانتقال من مرحلة استعراض المشروع داخل المنصة إلى مرحلة تشغيله الفعلي، دون الحاجة إلى إعدادات يدوية معقدة، الأمر الذي يساهم في تسهيل عملية اختبار المشاريع والتحقق من جاهزيتها قبل الانتقال إلى مراحل التحليل أو التحسين باستخدام أدوات الذكاء الاصطناعي.



الشكل 3-4 تشغيل مشروع الكرة الأرضية ثلاثية الأبعاد (D Earth Globe3) داخل منصة EHCode.

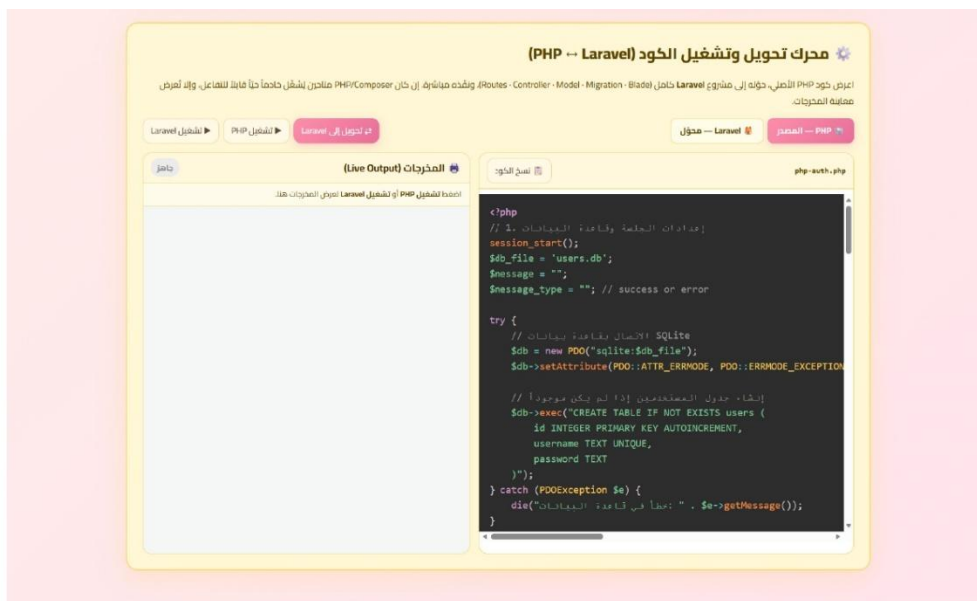
تعرض هذه الواجهة مشروع D Earth Globe3 بعد تشغيله من خلال منصة EHCode، والذي يمثل أحد المشاريع المولدة باستخدام نموذج GLM-5.1. ويهدف عرض هذا المشروع إلى توضيح قدرة النظام على تشغيل المشاريع التفاعلية التي تعتمد على الرسوميات ثلاثية الأبعاد، وإدارتها ضمن بيئة عمل موحدة. تظهر في أعلى الواجهة مجموعة من أدوات التحكم الخاصة بالمشروع، والتي تشمل زر تشغيل المشروع لبدء تنفيذ التطبيق، وزر الإيقاف لإيقاف التشغيل، وزر فتح نافذة جديدة لعرض المشروع في نافذة مستقلة، بالإضافة إلى زر دليل المشروع الذي يتيح الوصول إلى تعليمات التشغيل والاستخدام، وزر الإغلاق للعودة إلى قائمة المشاريع.

في الجزء الرئيس من الواجهة يتم عرض المشروع بعد تشغيله، حيث تظهر الكرة الأرضية ثلاثية الأبعاد مع لوحة تحكم تفاعلية تتيح تعديل سرعة دوران الكرة الأرضية، وإظهار أو إخفاء الغلاف الجوي والنجوم، والتبديل إلى الوضع الليلي، وإعادة ضبط المشهد إلى حالته الافتراضية. ويؤكد ذلك قدرة النظام على تشغيل المشاريع التفاعلية وعرضها بصورة مباشرة داخل المنصة دون الحاجة إلى استخدام أدوات خارجية أو تنفيذ أوامر يدوية.

توضح الأمثلة السابقة آلية عمل منصة EHCode في إدارة وتشغيل المشاريع البرمجية، حيث تدعم المنصة مجموعة متنوعة من المشاريع المولدة باستخدام نماذج الذكاء الاصطناعي، وتشمل تطبيقات الويب، ولوحات التحكم، والألعاب التعليمية، والمشاريع التفاعلية، والتطبيقات ثلاثية الأبعاد، إضافةً إلى العديد من المشاريع الأخرى التي تم اختبارها ضمن بيئة النظام. ويسهم هذا التنوع في التحقق من قدرة المنصة على إدارة وتشغيل مشاريع برمجية ذات طبيعة ومتطلبات مختلفة، مما يوفر أساساً متيناً للانتقال إلى عمليات التحليل والتحسين والمقارنة التي توفرها بقية مكونات النظام.

3-4 تحويل المشاريع من PHP إلى Laravel

تُعد ميزة تحويل المشاريع البرمجية من PHP التقليدية إلى إطار العمل Laravel إحدى الميزات الرئيسية التي توفرها منصة EHCode، حيث تعتمد على تقنيات الذكاء الاصطناعي للمساعدة في إعادة هيكلة المشروع وتحويله إلى مشروع متوافق مع معمارية Laravel. وتهدف هذه الميزة إلى تسهيل عملية تحديث المشاريع القديمة وتقليل الوقت والجهد اللازمين لإعادة تطويرها يدوياً، مع توفير بيئة موحدة لمراجعة الشيفرة البرمجية، وتنفيذ عملية التحويل، ومتابعة نتائجها مباشرةً داخل النظام.



الشكل 4-4 واجهة محرك تحويل المشاريع البرمجية من PHP إلى Laravel.

تعرض هذه الواجهة محرك تحويل المشاريع البرمجية في منصة EHCCode، والذي يتيح تنفيذ عملية تحويل مشاريع PHP التقليدية إلى مشاريع تعتمد على إطار العمل Laravel ضمن بيئة عمل موحدة، مع توفير جميع الأدوات اللازمة لإدارة عملية التحويل ومتابعة نتائجها.

في أعلى الواجهة يظهر عنوان محرك التحويل الذي يوضح وظيفة الصفحة، وهي تحويل المشاريع البرمجية بين PHP وLaravel، بالإضافة إلى وصف مختصر يبين أن النظام قادر على إعادة تنظيم مكونات المشروع بما يتوافق مع بنية Laravel، مثل ملفات Routes وControllers وModels وMigrations وBlade Templates.

تحت العنوان مباشرة تظهر مجموعة من أدوات التحكم الرئيسة، والتي تتضمن:

- زر تشغيل Laravel، ويستخدم لتشغيل المشروع بعد اكتمال عملية التحويل والتأكد من جاهزية ملفات المشروع.
 - زر تشغيل PHP، ويستخدم لتشغيل المشروع بصيغته الأصلية قبل تنفيذ عملية التحويل، بهدف مقارنة سلوك المشروع قبل وبعد التحويل.
 - زر تحويل إلى Laravel، وهو المسؤول عن بدء عملية التحويل، حيث يقوم النظام بتحليل المشروع وإعادة تنظيم بنيته البرمجية بما يتوافق مع إطار العمل Laravel.
- وفي الجهة اليمنى من شريط الأدوات تظهر أدوات اختيار الملفات، والتي تتضمن:
- قائمة اختيار الملف المصدر (PHP)، والتي تتيح تحديد الملف البرمجي المراد تحويله.
 - قائمة اختيار ملف Laravel الناتج، والتي تسمح بعرض الملف المقابل بعد انتهاء عملية التحويل، مما يسهل مراجعة المخرجات الناتجة عن النظام.

يتكون الجزء الرئيس من الواجهة من نافذتين تعملان بصورة متزامنة:

أولاً: نافذة محرر الشيفرة البرمجية

تقع في الجهة اليمنى، وتعرض محتوى الملف البرمجي المحدد قبل تنفيذ عملية التحويل. ويتيح هذا المحرر للمستخدم استعراض الشيفرة البرمجية ومراجعتها، مع المحافظة على تنسيق الكود وإبراز الكلمات المفتاحية البرمجية، الأمر الذي يسهل فهم بنية الملف قبل البدء بعملية التحويل.

ثانياً: نافذة المخرجات (Live Output)

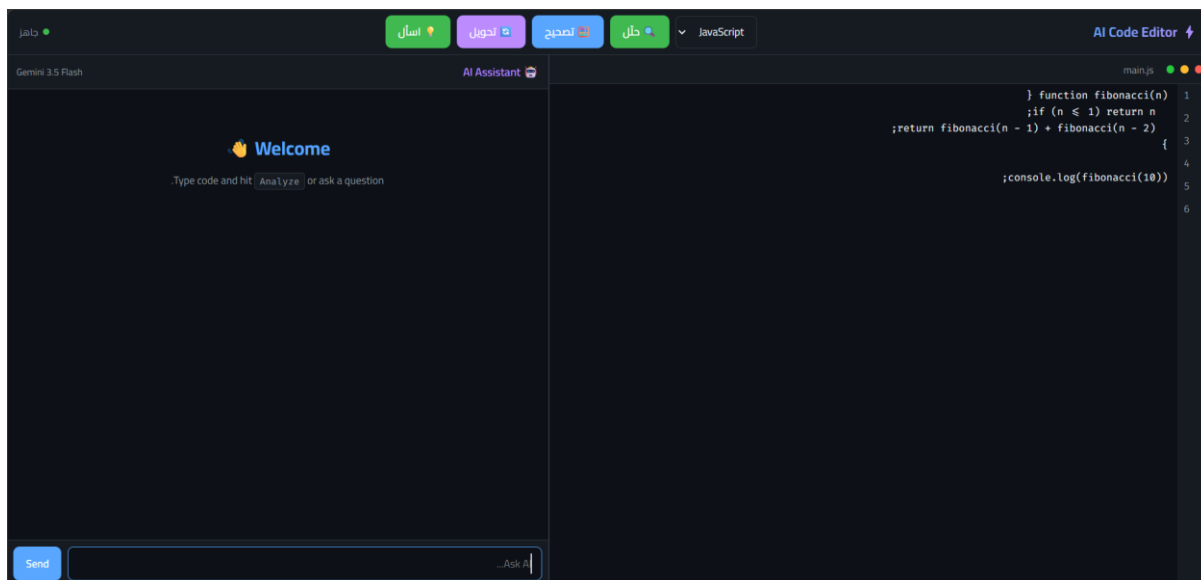
تقع في الجهة اليسرى، وتعرض الرسائل الناتجة عن تنفيذ العمليات المختلفة داخل النظام، مثل بدء تشغيل المشروع، أو تنفيذ عملية التحويل، أو تشغيل مشروع Laravel بعد اكتمال التحويل، بالإضافة إلى رسائل النجاح أو الأخطاء إن وجدت. وتساعد هذه النافذة المستخدم على متابعة جميع مراحل التنفيذ بصورة مباشرة دون الحاجة إلى استخدام نافذة أوامر خارجية.

ويتيح هذا التصميم تنفيذ جميع خطوات التحويل ضمن واجهة واحدة، بدءاً من اختيار الملف البرمجي، مروراً بمراجعة الشيفرة، ثم تنفيذ عملية التحويل وتشغيل المشروع، وانتهاءً بمراقبة نتائج التنفيذ والتحقق من نجاح العملية، مما يوفر بيئة تطوير متكاملة تُسهل في تبسيط عملية ترحيل المشاريع البرمجية إلى إطار العمل Laravel مع تقليل التدخل اليدوي وتحسين كفاءة سير العمل.

4-4 محرر الأكواد الذكي

يوفر نظام EHCode محرر أكواد ذكياً مدعوماً بتقنيات الذكاء الاصطناعي، يتيح للمستخدم تنفيذ مختلف عمليات معالجة الشيفرات البرمجية ضمن بيئة عمل واحدة. ولا يقتصر دور المحرر على كتابة وتحرير الشيفرة البرمجية، بل يوفر أيضاً أدوات متقدمة لتحليل الأكواد، واكتشاف المشكلات البرمجية، واقتراح

التحسينات، وتحويل الشيفرات بين لغات البرمجة المختلفة باستخدام نماذج الذكاء الاصطناعي، مما يجعله أداة متكاملة تساعد المطورين على تطوير البرمجيات بكفاءة أعلى.



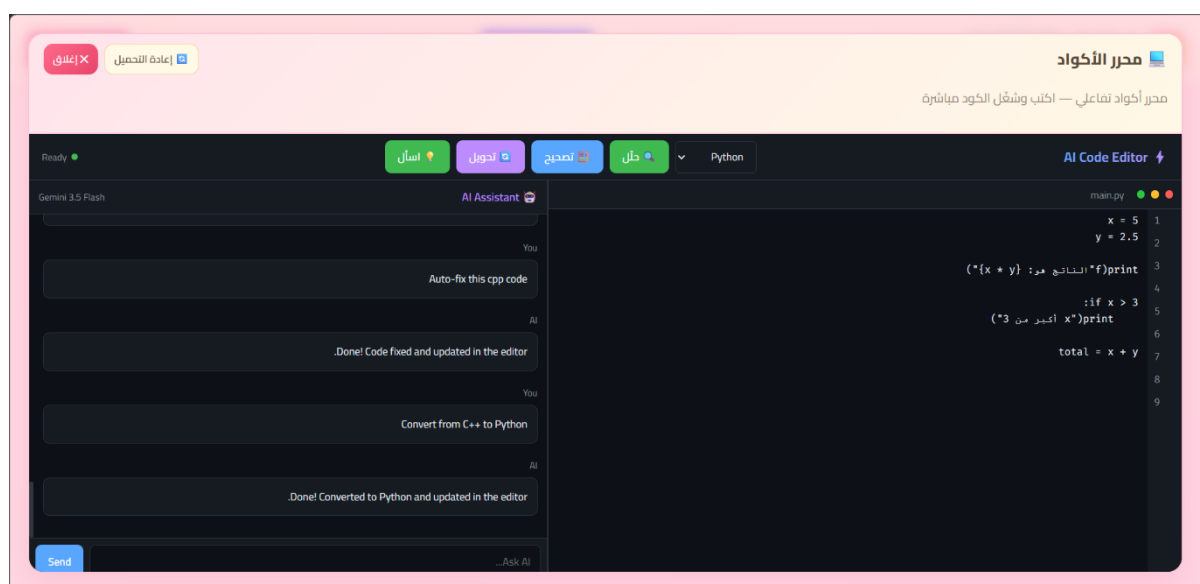
الشكل 4-5 الواجهة الرئيسية لمحرر الأكواد الذكي المدعوم بالذكاء الاصطناعي.

تعرض هذه الواجهة محرر الأكواد الذكي في منصة EHCCode، والذي يوفر بيئة تطوير متكاملة لكتابة الشيفرات البرمجية، وتحليلها، وتحويلها باستخدام تقنيات الذكاء الاصطناعي. وقد صُممت الواجهة بحيث تجمع بين محرر الأكواد والمساعد الذكي ضمن نافذة واحدة، مما يتيح تنفيذ مختلف العمليات البرمجية دون الحاجة إلى الانتقال بين أدوات أو برامج متعددة.

في أعلى الواجهة يظهر شريط أدوات المحرر الذي يضم مجموعة من الوظائف الرئيسة، وتشمل:

- قائمة اختيار لغة البرمجة، والتي تتيح تحديد اللغة المستخدمة في الشيفرة البرمجية، مثل JavaScript أو Python أو غيرها من اللغات المدعومة، ليتم تنفيذ عمليات التحليل أو التحويل بما يتوافق مع اللغة المختارة.
- زر حل (Solve)، ويستخدم لمعالجة المشكلات البرمجية أو المساعدة في إيجاد حلول للخوارزميات والمسائل البرمجية اعتمادًا على الذكاء الاصطناعي.

- زر تصحيح (Fix)، والذي يهدف إلى اكتشاف الأخطاء البرمجية واقتراح التعديلات اللازمة لتصحيحها وتحسين جودة الشيفرة.
 - زر تحويل (Convert)، ويستخدم لتحويل الشيفرة البرمجية من لغة برمجية إلى أخرى باستخدام نماذج الذكاء الاصطناعي مع المحافظة على الوظيفة المنطقية للكود.
 - زر اسأل (Ask)، والذي يتيح للمستخدم إرسال استفسارات مباشرة إلى المساعد الذكي للحصول على شرح للكود أو اقتراحات تطوير أو حلول للمشكلات البرمجية.
- في الجهة اليسرى من الواجهة تظهر منطقة المساعد الذكي (AI Assistant)، والتي تمثل مساحة التفاعل بين المستخدم ونموذج الذكاء الاصطناعي. وتعرض هذه المنطقة الرسائل والاستفسارات والإجابات، كما تحتوي في أسفلها على حقل لإدخال الأسئلة أو الطلبات، بالإضافة إلى زر Send لإرسالها إلى المساعد.
- أما في الجهة اليمنى فتظهر منطقة تحرير الشيفرة البرمجية، والتي تمثل المساحة الرئيسة لكتابة الكود أو تعديله. ويتميز المحرر بعرض أرقام الأسطر لتسهيل مراجعة الشيفرة وتتبع مواضع الأخطاء، بالإضافة إلى تمييز بناء الجملة (Syntax Highlighting)، مما يساهم في تحسين وضوح الكود وسهولة قراءته أثناء التطوير.
- كما تعرض الواجهة في الزاوية العلوية اليسرى حالة الاتصال بالنظام، والتي تشير إلى جاهزية المحرر للعمل، إضافةً إلى اسم نموذج الذكاء الاصطناعي المستخدم في تنفيذ العمليات البرمجية، مما يوضح للمستخدم النموذج المسؤول عن تحليل الشيفرة ومعالجة الطلبات.
- يوفر هذا التصميم بيئة تطوير موحدة تجمع بين تحرير الشيفرات البرمجية والتفاعل مع الذكاء الاصطناعي ضمن واجهة واحدة، الأمر الذي يساهم في تسريع عملية التطوير، وتحسين جودة الأكواد، وتقليل الوقت اللازم لتنفيذ المهام البرمجية المختلفة.



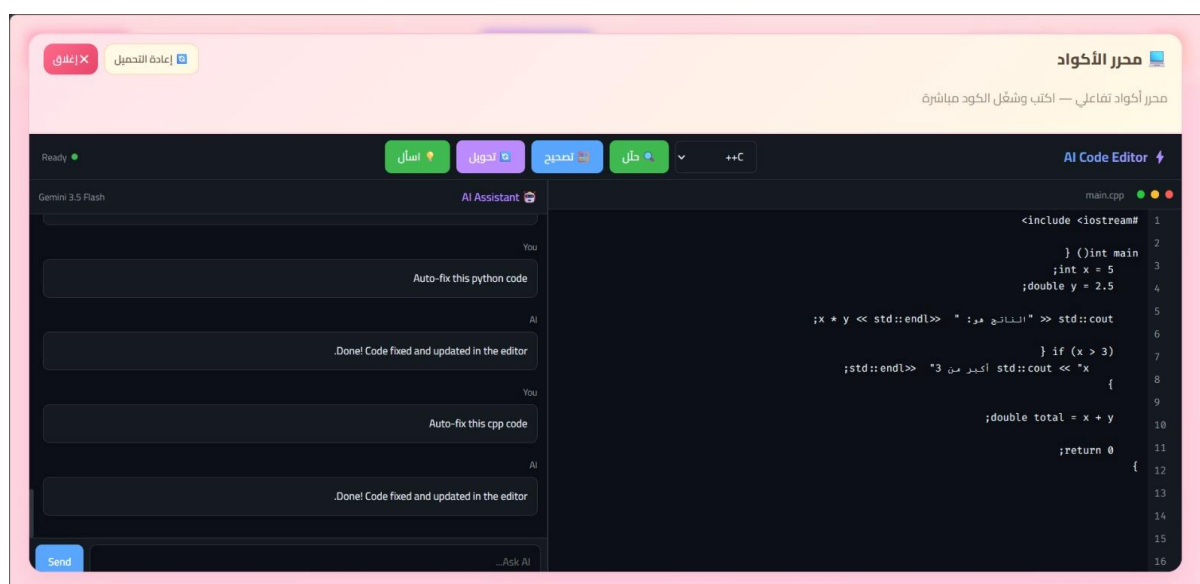
الشكل 4-6 كتابة الشيفرة البرمجية داخل محرر الأكواد الذكي

تعرض هذه الواجهة عملية كتابة الشيفرة البرمجية داخل محرر الأكواد الذكي في منصة EHCCode، حيث يوفر المحرر بيئة تطوير متكاملة تتيح للمستخدم إنشاء الشيفرات البرمجية وتعديلها بصورة مباشرة قبل تنفيذ عمليات التحليل أو التحويل.

تظهر في أعلى الواجهة أدوات التحكم الرئيسية للمحرر، والتي تشمل اختيار لغة البرمجة، بالإضافة إلى أزرار حل وتصحيح وتحويل واسأل، والتي تتيح الاستفادة من إمكانيات الذكاء الاصطناعي أثناء عملية التطوير.

في الجهة اليمنى يظهر محرر الشيفرة البرمجية الذي يعرض الكود المكتوب بلغة Python مع المحافظة على تنسيق الشيفرة وإظهار أرقام الأسطر، مما يسهل مراجعة الكود وتعديله. أما في الجهة اليسرى فتظهر نافذة المساعد الذكي، والتي تعرض سجل التفاعل بين المستخدم والذكاء الاصطناعي، بما في ذلك أوامر التصحيح والتحويل والرسائل التي توضح نجاح تنفيذ العمليات وتحديث الشيفرة داخل المحرر.

وتوضح هذه الواجهة كيفية دمج بيئة تحرير الأكواد مع أدوات الذكاء الاصطناعي ضمن واجهة واحدة، بما يسمح للمستخدم بكتابة الكود وإدارته دون الحاجة إلى استخدام أدوات خارجية.



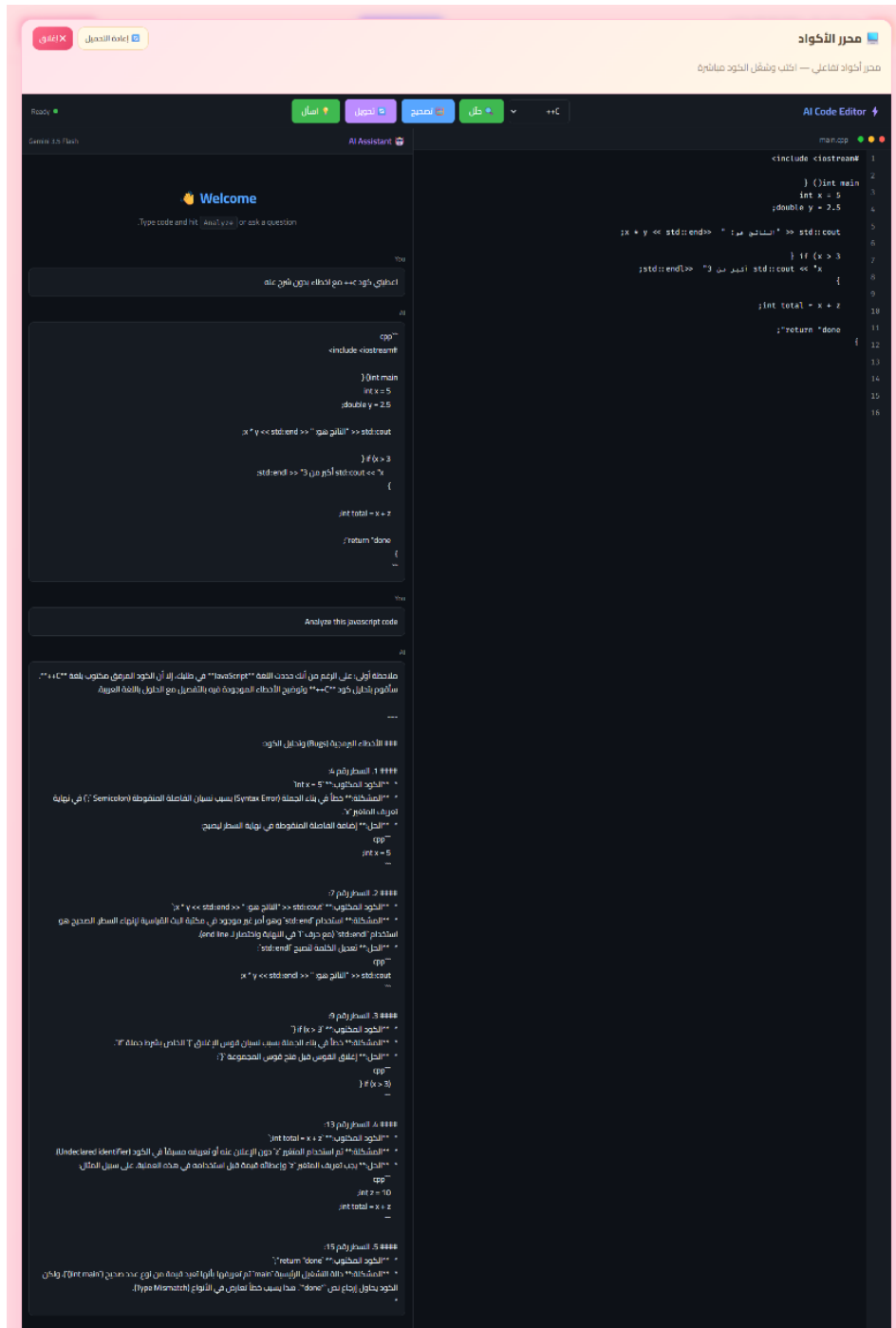
الشكل 4-7 تحويل الشيفرة البرمجية من لغة Python إلى لغة C++ باستخدام الذكاء الاصطناعي.

تعرض هذه الواجهة تنفيذ عملية تحويل الشيفرة البرمجية من لغة Python إلى لغة C++ باستخدام تقنيات الذكاء الاصطناعي داخل محرر الأكواد الذكي.

يبدأ المستخدم باختيار لغة المصدر ولغة الهدف، ثم تنفيذ عملية التحويل من خلال زر تحويل، ليقوم النظام بتحليل الشيفرة البرمجية وإعادة توليدها باللغة المطلوبة مع المحافظة على منطق التنفيذ والوظائف الأساسية للكود.

توضح الواجهة نجاح عملية التحويل، حيث يظهر الكود الناتج بلغة C++ داخل محرر الأكواد، بينما تعرض نافذة المساعد الذكي رسائل توضح تنفيذ عملية التحويل وتحديث محتوى المحرر تلقائياً، الأمر الذي يتيح للمستخدم مراجعة الشيفرة الجديدة مباشرة بعد انتهاء العملية.

وتبرز هذه الميزة قدرة النظام على تسهيل الانتقال بين لغات البرمجة المختلفة، بما يساعد المطورين على إعادة استخدام الخوارزميات وتطوير التطبيقات في بيئات برمجية متعددة دون الحاجة إلى إعادة كتابة الشيفرة يدوياً.



الشكل 8-4 تحليل الشيفرة البرمجية باستخدام الذكاء الاصطناعي

تعرض هذه الواجهة تنفيذ عملية تحليل الشيفرة البرمجية باستخدام الذكاء الاصطناعي داخل محرر الأكواد الذكي، حيث يقوم المستخدم بإرسال طلب التحليل من خلال زر حل ليبدأ النظام بفحص الشيفرة البرمجية وتقييمها.

يعرض المساعد الذكي في الجهة اليسرى تقريراً تفصيلياً يتضمن شرحاً لوظيفة الشيفرة البرمجية، وتحليلاً لآلية عملها، وتوضيحاً لتسلسل تنفيذ التعليمات، بالإضافة إلى ملاحظات تتعلق بجودة الكود، وأفضل الممارسات البرمجية، والاقتراحات التي تساعد على تحسين الأداء ورفع قابلية الصيانة.

وفي الجهة اليمنى تبقى الشيفرة البرمجية معروضة داخل المحرر، مما يسمح للمستخدم بمراجعة التقرير ومقارنته مع الكود في الوقت نفسه، دون الحاجة إلى التنقل بين نوافذ مختلفة.

وتسهم هذه الميزة في مساعدة المطورين على فهم الشيفرات البرمجية بصورة أفضل، واكتشاف نقاط الضعف والفرص المتاحة لتحسين جودة الكود، الأمر الذي يجعل محرر الأكواد الذكي أداة متكاملة لدعم عملية تطوير البرمجيات بالاعتماد على تقنيات الذكاء الاصطناعي.

5-4 المساعد البرمجي الذكي EHCode

يوفر نظام EHCode مساعداً برمجياً ذكياً يعتمد على نماذج الذكاء الاصطناعي لمساعدة المستخدمين في تنفيذ مختلف المهام البرمجية، مثل الإجابة عن الاستفسارات، وشرح الشيفرات البرمجية، واقتراح الحلول، وتحليل الأكواد، وتنفيذ التعليمات البرمجية من خلال واجهات استخدام متعددة. وقد تم توفير المساعد بنسختين، الأولى من خلال واجهة ويب تفاعلية، والثانية من خلال واجهة سطر الأوامر (CLI)، بما يمنح المستخدم مرونة في اختيار أسلوب التفاعل المناسب لطبيعة عمله.



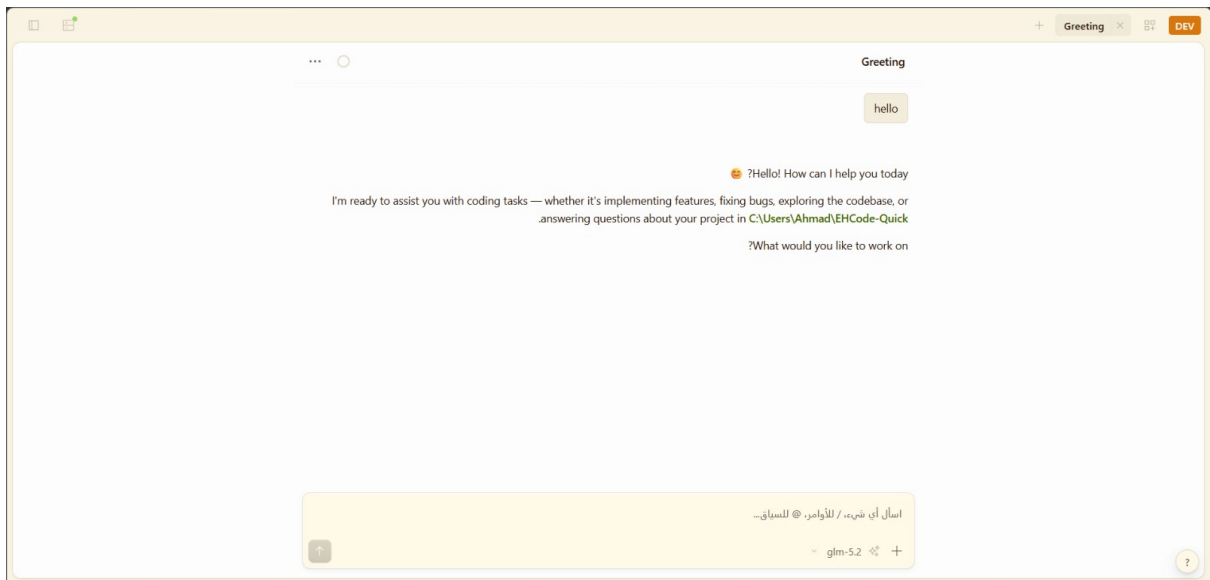
الشكل 4-9 الواجهة الرئيسية للمساعد البرمجي الذكي EHCCode Web.

تعرض هذه الواجهة الصفحة الرئيسية للمساعد البرمجي الذكي EHCCode Web، والتي تمثل نقطة البداية للتفاعل مع نماذج الذكاء الاصطناعي داخل النظام. وقد صُممت الواجهة بأسلوب بسيط يركز على تسهيل بدء جلسة جديدة وإرسال الطلبات البرمجية بسرعة، مع تقليل العناصر غير الضرورية لزيادة تركيز المستخدم على عملية التفاعل.

في أعلى الواجهة يظهر شريط التحكم الرئيسي، والذي يتضمن مجموعة من الأدوات الخاصة بإدارة الجلسات، حيث يوفر زر "جلسة جديدة" لإنشاء محادثة جديدة مع المساعد، كما يتيح للمستخدم إدارة الجلسة الحالية والانتقال بين الجلسات المختلفة عند الحاجة، بالإضافة إلى عرض بيئة التشغيل الحالية من خلال مؤشر DEV الموجود في الزاوية العلوية.

يتوسط الصفحة حقل إدخال الطلبات (Prompt)، والذي يمثل العنصر الرئيس في الواجهة، حيث يمكن للمستخدم إدخال مختلف أنواع الطلبات البرمجية، مثل إنشاء الأكواد، وشرح الشيفرات، وتحليل المشاريع، وتصحيح الأخطاء، أو تنفيذ أي مهمة تعتمد على الذكاء الاصطناعي.

أسفل حقل الإدخال تظهر قائمة اختيار نموذج الذكاء الاصطناعي، والتي تتيح للمستخدم تحديد النموذج الذي سيعالج الطلب قبل إرساله، مما يمنح مرونة في اختيار النموذج المناسب وفق طبيعة المهمة المطلوبة. كما تحتوي الواجهة على خيار EHCCode-Quick، والذي يوفر وضعًا سريعًا للتفاعل مع المساعد البرمجي، بهدف تسهيل تنفيذ المهام المتكررة وتقليل الزمن اللازم لإرسال الطلبات. وفي أسفل حقل الإدخال يظهر زر إرسال الطلب، والذي يستخدم لإرسال الاستفسار أو الأمر إلى نموذج الذكاء الاصطناعي والانتقال إلى واجهة المحادثة التي تعرض الاستجابة الناتجة. ويُسهّم هذا التصميم في توفير نقطة انطلاق بسيطة ومنظمة للتعامل مع المساعد البرمجي، حيث يستطيع المستخدم بدء جلسة جديدة، واختيار النموذج المناسب، وإرسال مختلف الطلبات البرمجية من خلال واجهة موحدة وسهلة الاستخدام.



الشكل 4-10 واجهة المحادثة مع المساعد البرمجي الذكي في منصة EHCCode Web.

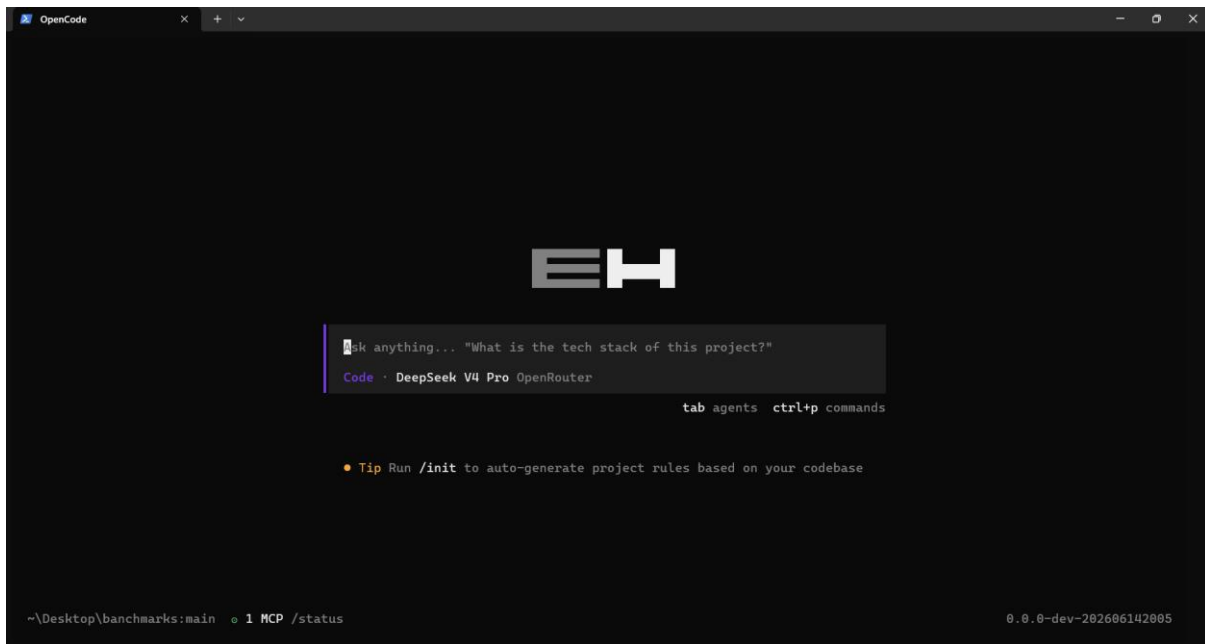
تعرض هذه الواجهة جلسة محادثة نشطة مع المساعد البرمجي الذكي EHCCode Web بعد إنشاء جلسة جديدة وإرسال أول طلب إلى نموذج الذكاء الاصطناعي. وتمثل هذه الواجهة البيئة الرئيسة للتفاعل بين المستخدم والمساعد، حيث يتم تبادل الطلبات والاستجابات بصورة متسلسلة ضمن جلسة واحدة.

في الجزء الأوسط من الواجهة يظهر سجل المحادثة، والذي يعرض جميع الرسائل المتبادلة بين المستخدم والمساعد البرمجي بترتيب زمني، مما يتيح للمستخدم متابعة سير المحادثة والرجوع إلى الإجابات السابقة عند الحاجة. كما يميز النظام بين رسائل المستخدم واستجابات المساعد لتسهيل متابعة الحوار وفهم تسلسل الطلبات.

في أسفل الواجهة يظهر حقل إدخال الطلبات، والذي يسمح للمستخدم بإدخال استفسارات جديدة أو أوامر برمجية إضافية، مثل طلب إنشاء شيفرة برمجية، أو شرح جزء من الكود، أو تحليل مشروع، أو اقتراح تحسينات، أو معالجة الأخطاء البرمجية. كما يتضمن الحقل قائمة اختيار نموذج الذكاء الاصطناعي لإرسال الطلب إلى النموذج المحدد، بالإضافة إلى زر الإرسال الذي يبدأ عملية معالجة الطلب وإضافة الاستجابة الجديدة إلى سجل المحادثة.

وفي أعلى الواجهة تظهر معلومات الجلسة الحالية، والتي توضح اسم المحادثة النشطة، مع إمكانية إنشاء جلسات جديدة أو التنقل بين الجلسات المختلفة، مما يساعد على تنظيم المحادثات وفق المشاريع أو المهام البرمجية المختلفة.

توفر هذه الواجهة بيئة تفاعلية متكاملة تسمح للمستخدم بإجراء حوار مستمر مع المساعد البرمجي، حيث يمكن إرسال سلسلة من الطلبات المتتابعة مع الاحتفاظ بسياق المحادثة، الأمر الذي يساهم في تنفيذ المهام البرمجية المعقدة بصورة أكثر كفاءة، ويجعل المساعد أداة فعالة لدعم عمليات التطوير وتحليل الشيفرات البرمجية داخل منصة EHCCode.



الشكل 4-11 واجهة المساعد البرمجي EHCodE ضمن بيئة سطر الأوامر (CLI).

تعرض هذه الواجهة المساعد البرمجي EHCodE أثناء تشغيله ضمن بيئة سطر الأوامر (Command Line Interface - CLI)، والتي توفر للمطورين وسيلة سريعة وفعالة للتفاعل مع نماذج الذكاء الاصطناعي مباشرةً من داخل بيئة التطوير، دون الحاجة إلى استخدام واجهة رسومية.

في منتصف الواجهة يظهر حقل إدخال الأوامر (Prompt)، والذي يمثل نقطة التفاعل الرئيسية مع المساعد، حيث يمكن للمستخدم كتابة مختلف الطلبات البرمجية، مثل تحليل مشروع، أو شرح الشيفرات البرمجية، أو اقتراح تحسينات، أو إنشاء أكواد جديدة، أو تنفيذ مهام تطوير متنوعة بالاعتماد على الذكاء الاصطناعي.

أسفل حقل الإدخال تعرض الواجهة النموذج المستخدم في معالجة الطلبات، مما يوضح للمستخدم نموذج الذكاء الاصطناعي المسؤول عن تنفيذ المهمة الحالية. كما توفر الواجهة اختصارات للوصول إلى الوظائف الإضافية، مثل Agents لإدارة الوكلاء البرمجيين، و Commands لاستعراض الأوامر المتاحة، الأمر الذي يساهم في تسهيل استخدام المساعد داخل بيئة سطر الأوامر.

وفي الجزء السفلي من الواجهة تظهر معلومات بيئة العمل الحالية، والتي تتضمن مسار المشروع المفتوح، والفرع الحالي لنظام إدارة الإصدارات Git، بالإضافة إلى حالة الاتصال بالخدمات المساندة، مما يمنح المطور تصورًا واضحًا عن بيئة التطوير التي يعمل ضمنها قبل تنفيذ الأوامر.

كما تعرض الواجهة رسالة إرشادية (Tip) تقترح استخدام أمر `init/` لإنشاء قواعد عمل خاصة بالمشروع اعتمادًا على محتوى الشيفرة البرمجية، وهو ما يساعد المساعد البرمجي على فهم بنية المشروع وتقديم استجابات أكثر دقة وتوافقًا مع طبيعة التطبيق.

توفر هذه الواجهة بيئة عمل خفيفة وسريعة تلبي احتياجات المطورين الذين يفضلون العمل من خلال سطر الأوامر، حيث تتيح تنفيذ مختلف المهام البرمجية والتفاعل مع نماذج الذكاء الاصطناعي مباشرة داخل بيئة التطوير، مما يساهم في تسريع سير العمل ورفع كفاءة تطوير البرمجيات.

4-6 مقارنة أداء نماذج الذكاء الاصطناعي

اعتمدت منصة EHCCode على ثلاثة نماذج للذكاء الاصطناعي هي GLM-5.1 و DeepSeek V4 و MiMo V2.5 Pro لتوليد المشاريع البرمجية وتحليلها. ولتقييم أداء هذه النماذج بصورة موضوعية، تم إجراء مقارنة شاملة استندت إلى مجموعة من مؤشرات الأداء ومعايير جودة البرمجيات، إضافةً إلى مقارنة عملية لمخرجات مشروع موحد تم توليده باستخدام النماذج الثلاثة. وقد ساهمت هذه المقارنات في تحديد نقاط القوة والاختلاف بين النماذج واختيار النموذج الأكثر كفاءة لدعم العمليات البرمجية داخل النظام.



الشكل 12-4 واجهة مقارنة نماذج الذكاء الاصطناعي وفق مؤشرات الأداء ومعايير جودة البرمجيات.

تعرض هذه الواجهة صفحة مقارنة نماذج الذكاء الاصطناعي في منصة EHCCode، والتي تمثل المرحلة النهائية من عملية تقييم النماذج المستخدمة في توليد المشاريع البرمجية. وتوفر الواجهة مجموعة من

الجدول والمؤشرات التي تساعد على تحليل أداء النماذج بصورة كمية ونوعية، مما يمكن المستخدم من اختيار النموذج الأنسب وفق متطلبات المشروع البرمجي.

في الجزء العلوي من الواجهة يظهر جدول مقارنة النماذج الثلاثة، والذي يعرض مقارنة مباشرة بين نماذج GLM-5.1 و DeepSeek V4 Pro و MiMo V2.5 Pro اعتمادًا على مجموعة من المؤشرات العملية

المستخلصة من المشاريع التي تم توليدها أثناء الدراسة.

ويتضمن الجدول عددًا من معايير التقييم، من أهمها:

- عدد المشاريع المولدة لكل نموذج.
- مدى توفر ملفات التشغيل (start.bat) وجاهزيتها للتنفيذ.
- عدد المشاريع المطورة باستخدام Node.js.
- عدد المشاريع المطورة باستخدام Python.
- عدد المشاريع الثابتة (HTML/CSS/JavaScript).
- عدد مشاريع Flutter.
- متوسط عدد الملفات داخل المشروع.
- توفر ملف README الخاص بتوثيق المشروع.
- التحقق من سلامة ملفات التشغيل وخلوها من الأخطاء.

وأظهرت نتائج المقارنة أن النماذج الثلاثة تمكنت من توليد 20 مشروعًا برمجيًا لكل نموذج، كما حققت جميعها نسبة نجاح كاملة في توفير ملفات التشغيل (20/20)، مما يدل على جاهزية جميع المشاريع للتنفيذ المباشر دون الحاجة إلى تعديلات إضافية.

كما بينت النتائج وجود اختلاف في تنوع المشاريع التي تمكن كل نموذج من توليدها؛ حيث تمكن نموذج GLM-5.1 من إنتاج 3 مشاريع Node.js، ومشروع Python، و13 مشروعًا ثابتًا، ومشروع Flutter. أما نموذج DeepSeek V4 Pro فقد أنتج 3 مشاريع Node.js، ولم يولد مشاريع Python أو Flutter، بينما تميز بإنتاج 17 مشروعًا ثابتًا. في المقابل، تمكن نموذج MiMo V2.5 Pro من إنتاج 4 مشاريع Node.js، ومشروع Python، و12 مشروعًا ثابتًا، ومشروع Flutter، مما يعكس اختلافًا في طبيعة المشاريع التي ركز عليها كل نموذج.

وأظهرت النتائج أيضًا اختلافًا في متوسط عدد الملفات داخل المشاريع، حيث بلغ المتوسط 3010 ملفات في مشاريع GLM-5.1، و3000 ملف في مشاريع DeepSeek V4 Pro، و3201 ملفًا في مشاريع MiMo V2.5 Pro، كما وفر نموذج GLM-5.1 خمسة ملفات README، مقابل ملف واحد في DeepSeek V4 Pro، وملفين في MiMo V2.5 Pro، وهو ما يعكس تفاوتًا في مستوى توثيق المشاريع البرمجية.

كما أوضحت نتائج التحقق من ملفات التشغيل أن جميع ملفات start.bat كانت سليمة وخالية من الأخطاء في النماذج الثلاثة، مما يؤكد جاهزية المشاريع للتشغيل مباشرة.

أسفل الجدول يظهر قسم النموذج الفائز، والذي يعرض النموذج الذي حقق أفضل النتائج بعد تحليل جميع مؤشرات الأداء، إضافةً إلى النموذج الموصى باستخدامه في المشاريع البرمجية، إلى جانب مجموعة من الملاحظات العامة التي تلخص أهم النتائج المستخلصة من عملية التقييم.

وأظهرت نتائج التقييم أن نموذج GLM-5.1 حقق أفضل أداء عام، ولذلك تم اختياره بوصفه النموذج الفائز في الدراسة، كما أوصت المنصة باستخدامه في المشاريع البرمجية العامة، نظرًا لما حققه من توازن بين جودة المخرجات، وتنوع المشاريع، وتوافقها مع متطلبات التشغيل. كما بينت النتائج إمكانية الاستفادة

من نموذج DeepSeek V4 Pro في المشاريع التي تتطلب تركيزًا أكبر على تطوير الواجهة الخلفية (Backend)، بينما قدم نموذج MiMo V2.5 Pro أداءً جيدًا في المشاريع التي تعتمد على تنظيم المحتوى وتصميم الواجهات.

وفي الجزء السفلي من الواجهة يظهر جدول مقارنة معايير الجودة في هندسة البرمجيات، والذي يعتمد على مجموعة من معايير الجودة المستندة إلى مبادئ هندسة البرمجيات لتقييم جودة المشاريع الناتجة من منظور هندسي، وليس فقط من حيث إمكانية تشغيلها.

ويتضمن الجدول المعايير التالية:

- الجودة الوظيفية (Functional Quality).
- الأداء (Performance).
- الاعتمادية (Reliability).
- الأمان (Security).
- سهولة الاستخدام (Usability).
- قابلية الصيانة (Maintainability).
- كفاءة التكلفة (Cost Efficiency).
- قابلية النقل (Portability).

كما يوضح الجدول الوزن النسبي لكل معيار، إضافةً إلى الدرجة التي حصل عليها كل نموذج، مع استخدام مؤشرات بصرية تسهل المقارنة المباشرة بين النتائج.

وأظهرت نتائج تقييم الجودة أن نموذج GLM-5.1 حقق أعلى درجة في الجودة الوظيفية (10/9.2) وسهولة الاستخدام (10/9.0) وقابلية الصيانة (10/9.2)، بينما حقق نموذج DeepSeek V4 Pro أفضل

النتائج في الأداء (10/9.2) والاعتمادية (10/9.0) والأمان (10/8.8) وكفاءة التكلفة (10/9.5). أما نموذج MiMo V2.5 Pro فقد سجل أفضل نتيجة في قابلية النقل (10/9.0) مع أداء متوازن في بقية المعايير.

وبعد احتساب الأوزان النسبية لجميع معايير الجودة، حقق نموذج GLM-5.1 أعلى مجموع موزون بلغ 10/8.78، يليه نموذج DeepSeek V4 Pro بمجموع 10/8.75، ثم نموذج MiMo V2.5 Pro بمجموع 10/7.96، وهو ما يؤكد تفوق نموذج GLM-5.1 في التقييم النهائي، واعتماده بوصفه النموذج الأكثر كفاءة لتوليد وتحليل المشاريع البرمجية داخل منصة EHCCode.

الفصل الخامس: النتائج والآفاق

المستقبلية

خلصت هذه الدراسة إلى تقديم منصة متكاملة تحت مسمى EHCode Hub، تم تصميمها وبنائها بهدف تقييم ومقارنة قدرات النماذج اللغوية الكبيرة في توليد الأكواد البرمجية، حيث طبقت المنصة على ثلاثة نماذج رائدة هي GLM-5.1 من Zhipu AI، و DeepSeek V4 Pro، و MiMo V2.5 Pro من Xiaomi. وقد تم تصميم عشرين مهمة برمجية موزعة على عشر فئات تطويرية رئيسية شملت تطبيقات الويب المتكاملة، وواجهات برمجة التطبيقات، والتطبيقات الواجهية، وتطبيقات بايثون، والتجارة الإلكترونية، وتصور البيانات، والنمذجة ثلاثية الأبعاد، وتطبيقات الجوال باستخدام Flutter، وأدوات الأتمتة، وأنتجت النماذج الثلاثة مجتمعة ستين مشروعاً بتنوع تقني ثري عكس قدرات كل نموذج في التعامل مع مختلف أنواع التطبيقات.

تم بناء خادم عارض تفاعلي باستخدام تقنية Node.js معتمداً حصراً على وحداته المدمجة وفق مبدأ "صفر اعتماديات"، مما يضمن سهولة النقل والتشغيل على أي جهاز دون الحاجة إلى تثبيت اعتماديات إضافية، كما تم تطوير واجهة أمامية تفاعلية باستخدام HTML5 و CSS3 و JavaScript الخام، تمكن المستخدم من استعراض المشاريع، وتشغيلها تلقائياً مع إدارة المنافذ والعمليات بشكل آلي بالكامل، وعرضها داخل نافذة iframe، والاطلاع على دليل توثيقي تفصيلي لكل مشروع، ومقارنة النماذج عبر جدول مقارنة شامل يستعرض مؤشرات الأداء الرئيسية ويصدر توصية تلقائية بالنموذج الأنسب. كما تم دمج مساعد EHCode (opencode) داخل المنصة عبر iframe، مما جعل المركز واجهة موحدة تجمع بين التقييم والمساعدة البرمجية الذكية في مكان واحد، مع توفير مشغل موحد على سطح المكتب يقلع جميع المكونات في الخلفية بنقرة واحدة، ومشغل إيقاف لتنظيف الموارد وإعادة ضبط المنافذ بسهولة.

أظهرت النتائج تفوقاً واضحاً لنموذج GLM-5.1 من حيث الالتزام الكامل بالمواصفات الفنية، حيث أنتج تطبيقات Flutter حقيقية كما هو مطلوب في المهام المخصصة للجوال، وسجل أعلى نسبة توثيق بخمسة

مشاريع احتوت على ملف README من أصل عشرين، مما جعله النموذج الموصى به للاستخدام العام في توليد الأكواد البرمجية. في المقابل، أظهر نموذج DeepSeek V4 Pro انحرافاً ملحوظاً عن المتطلبات من خلال إنتاجه صفحات HTML بدلاً من تطبيقات Flutter في المهمتين المخصصتين للجوال، كما كان الأضعف في التوثيق بمشروع واحد فقط، بينما أظهر نموذج MiMo V2.5 Pro أداءً متوازناً في المشاريع الواجهية والجوال، مع التزام كامل بالمواصفات وتوثيق متوسط. أثبتت المنصة كفاءة عالية في إدارة العمليات وتحرير المنافذ، ونجح دمج المساعد البرمجي دون عوائق أمنية بفضل غياب ترويسات منع التضمين، كما أثبتت البنية المعمارية القائمة على الفصل بين الاهتمامات وصفر اعتماديات جدواها في سهولة النقل والتشغيل، وشكلت المخططات المصممة بلغة PlantUML توثيقاً بصرياً دقيقاً لهيكلية النظام وسلوكه الديناميكي، وقدم التصميم المعماري نموذجاً مرناً قابلاً للتوسع يلبي متطلبات التقييم الحالية والمستقبلية.

يمثل هذا المشروع إسهاماً عملياً ملموساً في مجال تقييم النماذج اللغوية الكبيرة، ويوفر أداة مفتوحة المصدر يمكن للباحثين والمطورين الاستفادة منها في تقييم نماذج جديدة، واتخاذ قرارات مستنيرة بشأن النموذج الأنسب لتطبيقاتهم المختلفة، مع تعزيز الشفافية والموضوعية في عملية المقارنة، ودعم بيئة التطوير العربية عبر واجهة مستخدم عربية شاملة تجعل التقنيات المتقدمة في متناول المطورين والباحثين العرب.

يمكن تطوير المشروع في عدة اتجاهات مستقبلية لتعزيز قيمته العلمية والعملية وتوسيع نطاق استخدامه، حيث يُقترح توسيع قاعدة النماذج المُقيّمة بإضافة نماذج جديدة مثل GPT و Claude و Gemini و Qwen و Grok وغيرها، مما يجعل المنصة مرجعاً شاملاً لمقارنة جميع النماذج الرائدة في السوق، مع تحديث مجموعة المهام البرمجية لتشمل تحديات أكثر تعقيداً مثل تطبيقات الذكاء الاصطناعي وتحليل البيانات الضخمة وتطبيقات البلوكشين والأنظمة الموزعة، كما يُقترح تعزيز التكامل مع مساعد EHCode عبر

تمكين تمرير سياق المشروع المفتوح حالياً إلى المساعد تلقائياً، وتطوير آلية تواصل ثنائي الاتجاه بين العارض والمساعد باستخدام تقنية `postMessage`، وإضافة مؤشر حالة اتصال حيوي في الواجهة، وتوفير إمكانية طرح أسئلة حول كود المشروع المفتوح مباشرة، مما يحول المنصة إلى بيئة تطوير تعلم تفاعلية متكاملة. كذلك يمكن أتمتة عملية التقييم الكمي بتطوير نظام تقييم آلي يعتمد على معايير متعددة الأبعاد تشمل قابلية التشغيل الفعلية وجودة الكود (باستخدام أدوات تحليل جودة الكود) والجودة البصرية (عبر أدوات اختبار واجهات المستخدم الآلية) واكتمال المتطلبات، مع تخزين النتائج تلقائياً في ملف `results.json` دون تدخل يدوي، وإضافة لوحة تحكم إحصائية متقدمة تعرض الرسوم البيانية والتحليلات المقارنة.

أما على صعيد تحسين الأداء وتجربة المستخدم، فيمكن استخدام تقنية `WebSocket` بدلاً من آلية الاستقصاء (Polling) للحصول على تحديثات فورية لحالة المشاريع، وتطبيق آليات تخزين مؤقت (Caching) لنتائج فحص المشاريع لتسريع التحميل، وإضافة دعم لأنظمة التشغيل `Unix/Linux` و `macOS`، وتوفير واجهة مستخدم متجاوبة بالكامل مع جميع أحجام الشاشات، وإضافة خاصية البحث والتصفية للمشاريع والنماذج لتسهيل التنقل بين المشاريع الستين. كما يُوصى ببناء مجتمع حول المنصة عبر نشر المشروع كمصدر مفتوح على `GitHub` مع توثيق شامل للإسهام، وتوفير واجهات برمجة تطبيقات (APIs) مفتوحة تسمح للمطورين ببناء إضافات وأدوات متكاملة مع المنصة، وإضافة دعم للغات متعددة في الواجهة (الإنجليزية والفرنسية والصينية) لتعميم الفائدة، وتطوير نسخة سحابية من المنصة تتيح التقييم عن بُعد دون تثبيت محلي، مما يسهل على الباحثين والمطورين من مختلف أنحاء العالم استخدام المنصة دون قيود بيئية.

فضلاً عن ذلك، يمكن استكشاف تطبيقات تعليمية للمنصة باستخدامها كأداة تعليمية في دورات علوم الحاسوب وهندسة البرمجيات، لتعليم الطلاب كيفية تقييم الكود المولد آلياً، ومقارنة الأساليب البرمجية المختلفة، وفهم نقاط القوة والضعف في النماذج اللغوية الكبيرة، مع إضافة خاصية "التحدي" حيث يتنافس الطلاب مع النماذج في حل المهام البرمجية نفسها، مما يعزز مهاراتهم البرمجية وقدرتهم على التقييم النقدي للكود. وأخيراً، يُقترح ربط المنصة بمعايير مرجعية عالمية من خلال دمج معايير مثل HumanEval وMBPP وSWE-bench كطبقة إضافية من التقييم، مما يسمح بمقارنة أداء النماذج على المنصة مع نتائجها في هذه المعايير المرجعية المعترف بها عالمياً، ويعزز المصداقية الأكاديمية للمنصة ويجعلها أداة بحثية معتمدة في الأوساط الأكاديمية والصناعية على حد سواء..

- [1] أحمد بن سليمان، وخالد محمد المري. (2025). "النماذج اللغوية الكبيرة في توليد الأكواد البرمجية: دراسة تحليلية لأداء النماذج مفتوحة المصدر". مجلة الذكاء الاصطناعي التطبيقي، المجلد 12، العدد 3، الصفحات 45-67.
- [2] سارة عبدالله الناصر، وفيصل حمد القحطاني. (2024). "تحديات تقييم النماذج التوليدية في مهام البرمجة: مراجعة منهجية". المؤتمر الدولي السادس لهندسة البرمجيات والأنظمة الذكية، الصفحات 112-125.
- [3] محمد رشيد العتيبي، وليلى إبراهيم السالم. (2023). "منصات قياس الأداء لتقييم مخرجات الذكاء [3] المجلة العربية للحوسبة السحابية، المجلد HumanEval و MBPP الاصطناعي: دراسة مقارنة بين 8، العدد 2، الصفحات 88-102.
- [4] نورة خالد الدوسري، وعبدالله سعد الشمري. (2025). "جودة الكود المُولّد بواسطة الذكاء الاصطناعي: [4] مقاييس ومعايير تقييمية في البيئات الهندسية الحديثة". مجلة البرمجيات والابتكار، العدد 15، الصفحات 33-49.
- [5] تركي فهد العجلان، ومنيرة صالح الحربي. (2024). "دمج المساعدات البرمجية الذكية في بيئات [5] التطوير المتكاملة: فرص وتحديات". أبحاث الحوسبة التفاعلية، المجلد 11، العدد 4، الصفحات 210-226.
- [6] Chen, L., Guo, Q., Jia, H., Zeng, Z., Wang, X., Xu, Y., Wu, J., Wang, Y., Gao, Q., Wang, J., Ye, W., & Zhang, S. (2024). A Survey on Evaluating Large Language Models in Code Generation Tasks. Journal of Computer Science and Technology.
- [7] Surveying the Benchmarking Landscape of Large Language Models in Code Intelligence. (2025). ACM Transactions on Software Engineering and Methodology.
- [8] Bayram, A., et al. (2025). Comparative Analysis of AI Models for Python Code Generation: A HumanEval Benchmark Study. Applied Sciences, 15(18), 9907.
- [9] Zheng, Z., Ning, K., Wang, Y., Zhang, J., Zheng, D., Ye, M., & Chen, J. (2023). A Survey of Large Language Models for Code: Evolution, Benchmarking, and Future Trends. arXiv preprint arXiv:2311.10372.

- [10] Molzam Sharifloo, A., Heydari, M., Kazerooni, P., Maninger, D., & Mezini, M. (2025). Where Do LLMs Still Struggle? An In-Depth Analysis of Code Generation Benchmarks. arXiv preprint arXiv:2511.04355.
- [11] OpenCode.ai. (2026). وكيل برمجة بالذكاء الاصطناعي مفتوح المصدر. <https://opencode.ai/ar>
- [12] timpara. (2026). opencode-academic-research: OpenCode port of academic-research-skills. GitHub. <https://github.com/timpara/opencode-academic-research>
- [13] npm. (2026). opencode-research-papers. <https://www.npmjs.com/package/opencode-research-papers>
- [14] wesammustafa. (2026). opencode-primer: Master OpenCode, the open-source AI coding agent. GitHub. <https://github.com/wesammustafa/opencode-primer>
- [15] AI Research. (2025, December). Dialectical Autocoding: Adversarial Cooperation in LLM Agents. arXiv preprint.